

Plan completo de migración — Landing pública Debacu Evaluation360

1. Objetivo real de la migración

El objetivo no es solo cambiar el diseño de la página principal. El objetivo correcto es separar y modernizar la capa pública de Debacu sin romper la lógica actual de producción.

La nueva estructura debe conseguir cuatro cosas a la vez:

1. Mantener la lógica real ya operativa en producción.
 2. Sustituir la estética actual por una experiencia pública más SaaS y más profesional.
 3. Rehacer Login y Solicitud de acceso con el diseño nuevo, pero conectados al backend real.
 4. Mantener el bloque legal actual, incluido el modal del `LegalFooter`, porque ya resuelve una parte delicada que no conviene rehacer sin necesidad.
-

2. Decisión arquitectónica recomendada

2.1. Qué se mantiene

Se mantiene todo lo que ya esté funcionando en producción y tenga impacto legal, operativo o de negocio:

- Enrutado público actual basado en `react-router-dom`.
- Lógica real de login.
- Lógica real de solicitud de acceso.
- Persistencia en base de datos.
- Edge Functions actuales como base de trabajo.
- `LegalFooter` actual y su modal legal.
- Flujo de revisión manual por admin.
- Flujo posterior de invitación para creación de contraseña.

2.2. Qué se sustituye

Se sustituye la capa visual pública actual:

- Landing principal.
- Navegación pública.
- Páginas comerciales tipo SaaS.
- Contacto.

- Presentación de planes.
- Login visual.
- Solicitud de acceso visual.

2.3. Qué no conviene hacer

No conviene:

- Mezclar el diseño nuevo con el `PublicLanding` actual añadiendo parches.
- Meter toda la nueva navegación dentro de un único componente gigante.
- Cambiar backend y frontend a la vez sin mapear antes campos, rutas y payloads.
- Rehacer el área legal si el modal actual ya cubre esa necesidad correctamente.

La migración debe hacerse por capas.

3. Estado actual identificado

3.1. Lo que ya existe en producción

La landing actual `PublicLanding` ya incluye:

- Hero principal.
- CTA a login.
- CTA a solicitar-acceso.
- Planes actuales con modales de condiciones.
- Pie legal `LegalFooter`.
- Navegación simple.
- Parte visual y texto orientado a Debacu Evaluation360 como plataforma privada / restringida.

3.2. Lo que ya existe en el diseño nuevo

La nueva propuesta visual ya aporta:

- Header mucho más SaaS.
- Home más sólida visualmente.
- Secciones separadas: producto, tecnología, docs, planes, contacto.
- Login mucho más profesional.
- Solicitud de acceso mucho más profesional.
- Página de planes más presentable y mejor jerarquizada.

- Contacto más serio que el actual.

3.3. Lo que debe combinarse

La solución correcta es:

- **Diseño visual nuevo**
 - **Lógica funcional de producción actual**
 - **Footer legal actual**
 - **Backend actual extendido donde haga falta**
-

4. Arquitectura objetivo recomendada

4.1. Estructura frontend propuesta

```
src/  
  pages/  
    public/  
      LandingPage.tsx  
      ProductPage.tsx  
      TechnologyPage.tsx  
      DocsPage.tsx  
      PricingPage.tsx  
      ContactPage.tsx  
      LoginPage.tsx  
      AccessRequestPage.tsx  
  
  components/  
    public/  
      PublicNavbar.tsx  
      PublicFooter.tsx  
      HeroSection.tsx  
      ProblemSection.tsx  
      FeaturesSection.tsx  
      PlansSection.tsx  
      ContactModalOrPage.tsx  
      PlanComparisonTable.tsx  
  
  legal/  
    LegalFooter.tsx  
    ...actual legal modal components...
```

4.2. Rutas públicas objetivo

/	-> LandingPage
/producto	-> ProductPage
/tecnologia	-> TechnologyPage
/docs	-> DocsPage
/planes	-> PricingPage
/contacto	-> ContactPage
/login	-> LoginPage
/solicitar-acceso	-> AccessRequestPage

4.3. Integración del footer legal

El LegalFooter actual debe mantenerse y conectarse al nuevo diseño.

La forma correcta no es rehacer su contenido, sino:

- importarlo en las nuevas páginas públicas
- mantener el modal legal actual
- revisar solo estilos exteriores si hace falta para que visualmente encaje con el nuevo shell público

En otras palabras: el footer legal debe **llamarse desde la nueva capa pública**, no rediseñarse desde cero salvo que haya un motivo legal o UX fuerte.

5. Estrategia de migración por fases

Fase 1 — Inventario técnico real

Antes de tocar nada, hay que identificar exactamente:

1. Dónde se definen hoy las rutas públicas.
2. Qué componentes se usan hoy para login.
3. Qué componente se usa hoy para solicitar-acceso.
4. Qué función / servicio / edge function recibe la solicitud de acceso.
5. Qué campos manda hoy el frontend.
6. Qué tabla guarda hoy la solicitud.
7. Cómo se genera hoy el PDF de aceptación.
8. Cómo se guarda ese PDF.
9. Cómo revisa el admin la solicitud.
10. Cómo se lanza después la invitación para crear contraseña.

Sin ese inventario, migrar diseño y backend a la vez es una fuente segura de errores.

Fase 2 — Nueva shell pública sin tocar backend

Crear la nueva capa pública tipo SaaS usando `react-router-dom`, pero sin modificar todavía la lógica interna.

Entregables de esta fase

- `LandingPage.tsx`
- `PublicNavbar.tsx`

- páginas públicas nuevas
- integración del LegalFooter
- routing limpio

Resultado esperado

Tener ya una web pública nueva y profesional con navegación moderna, aunque login y solicitud todavía puedan estar parcialmente conectados a los componentes actuales.

Fase 3 — Migración visual de Login

Rehacer el diseño del login, pero sin cambiar la lógica de autenticación.

Qué debe mantenerse

- validación actual
- servicio actual de autenticación
- navegación post-login actual
- mensajes de error actuales si ya funcionan bien

Qué se cambia

- layout
- jerarquía visual
- copy
- tarjetas laterales o panel informativo
- responsive

Riesgo

Bajo, siempre que no se cambie la lógica real.

Fase 4 — Migración funcional de Solicitud de acceso

Esta es la fase crítica.

No es un simple cambio de diseño. Aquí cambia el contrato de datos.

Lo que cambia realmente

- más campos en el formulario
- más validación
- probablemente nuevo payload
- probablemente cambio en tabla BD

- seguramente cambio en Edge Function
- posible cambio en generación de PDF
- posible cambio en almacenamiento del PDF

Riesgo

Alto si se hace sin mapear el backend actual.

Fase 5 — Integración admin posterior

Una vez el nuevo formulario guarde bien la solicitud completa:

- validar cómo consume el admin esos datos
- revisar si la pantalla admin necesita ver nuevos campos
- confirmar el flujo de invitación
- confirmar estados de la solicitud

Fase 6 — Limpieza final

Cuando todo lo nuevo funcione:

- retirar PublicLanding antiguo
- eliminar duplicados
- unificar estilos
- revisar imports viejos
- limpiar rutas obsoletas

6. Decisiones funcionales recomendadas

6.1. Landing principal

La nueva landing debe ser claramente SaaS, pero con el posicionamiento correcto de Debacu.

No debe vender solo “incidencias” como hace la landing actual.

Debe posicionar Debacu como una plataforma más amplia:

- risk / incidencias
- revenue intelligence
- alertas operativas
- decisiones con datos
- uso profesional y acceso restringido

Recomendación

Mantener el tono profesional y restringido del producto actual, pero con un framing más fuerte de plataforma hotelera, no solo de control de incidencias.

6.2. Planes

La parte nueva de planes es mejor que la actual y debe adoptarse, pero no como un simple diseño. Debe adaptarse a la realidad comercial de Debacu.

Recomendación

Separar visualmente:

- plan SaaS mensual
- add-ons técnicos
- integraciones enterprise

Porque ahora mismo, si todo se mezcla, el mensaje comercial se vuelve confuso.

Estructura recomendada

1. Planes base SaaS.
2. Tabla comparativa.
3. Add-ons:
 - API
 - Integración PMS
4. Nota comercial clara:
 - alta controlada
 - no contratación inmediata automática si no quieres ese modelo

6.3. Contacto

El contacto actual es demasiado pobre para una web pública seria.

La versión nueva es más profesional y conviene usarla.

Recomendación

Decidir entre dos enfoques:

Opción A — contacto como formulario real

Guardar contactos / leads en BD y que llegue a email o panel admin.

Opción B — contacto como modal o formulario ligero

Solo para captar interés comercial básico.

Recomendación realista: empezar por A solo si ya tienes backend o capacidad inmediata para procesarlo; si no, empezar por B pero dejando clara la continuidad futura.

6.4. Footer legal

Debe mantenerse.

Solo hay que integrar visualmente:

- spacing
- fondo
- paddings
- anchuras

Pero no tocar el contenido legal ni el modal si ya funciona.

7. Migración específica de Login

7.1. Objetivo

Usar el nuevo diseño de login, conectándolo al backend real actual.

7.2. Qué debe reaprovecharse

- función de autenticación actual
- estado de loading
- manejo de errores
- redirección post-login
- recuperación de contraseña si ya existe

7.3. Qué debe revisarse

- si el login actual usa servicio interno o Supabase/Auth/etc.
- si hay remember me real o solo visual
- si el enlace “¿Has olvidado tu contraseña?” está ya operativo

7.4. Riesgos

- romper redirecciones
 - romper provider/context de auth
 - perder mensajes reales de error
-

8. Migración específica de Solicitud de acceso

8.1. Objetivo

Pasar de un formulario más simple a un formulario profesional completo, manteniendo el flujo legal y de validación manual.

8.2. Campos recomendados del nuevo formulario

Datos de contacto

- nombre
- apellidos
- cargo / puesto
- email profesional
- teléfono

Datos de empresa / alojamiento

- empresa / alojamiento
- razón social
- CIF / NIF
- tipo de establecimiento
- dirección
- ciudad
- provincia
- país
- número de habitaciones
- web

Datos operativos opcionales o recomendables

- PMS actual
- número estimado de usuarios
- módulos de interés
- observaciones

Declaraciones legales

- aceptación de términos y condiciones
- declaración de uso profesional

- aceptación expresa registrada
- fecha/hora de aceptación
- referencia del PDF generado

8.3. Flujo objetivo del formulario

1. El usuario rellena todos los campos.
2. Abre y acepta términos.
3. Se genera PDF justificante con fecha y datos.
4. El PDF se guarda en storage.
5. Se inserta la solicitud completa en BD.
6. Estado inicial = PENDING.
7. El admin revisa.
8. Si aprueba, se genera la invitación para crear contraseña.

8.4. Qué hay que revisar del backend actual

Hay que identificar exactamente:

- nombre de la Edge Function actual
- payload actual
- tabla actual
- columnas actuales
- validaciones actuales
- flujo actual del PDF
- bucket/storage usado
- cómo se vincula el PDF a la solicitud

8.5. Riesgos principales

- que el frontend mande campos que la Edge Function no soporta
 - que la tabla no tenga columnas nuevas
 - que el PDF actual no soporte los nuevos datos
 - que la pantalla admin no muestre la información nueva
 - que el flujo de invitación espere un esquema antiguo
-

9. Modelo objetivo de datos para solicitud de acceso

Esto es una propuesta de referencia. Debe ajustarse al esquema real de producción.

```
interface AccessRequestPayload {
  firstName: string;
  lastName: string;
  jobTitle?: string;
  professionalEmail: string;
  phone?: string;

  companyName: string;
  legalName?: string;
  taxId: string;
  establishmentType: string;
  addressLine1?: string;
  city?: string;
  province?: string;
  country?: string;
  roomCount?: number;
  website?: string;

  pmsName?: string;
  estimatedUsers?: number;
  requestedModules?: string[];
  notes?: string;

  termsAccepted: boolean;
  professionalUseAccepted: boolean;
  termsAcceptedAt: string;

  pdfStoragePath?: string;
  pdfUrl?: string;
}
```

Y en BD una estructura similar a:

```
interface AccessRequestRecord {
  id: string;
  status: "PENDING" | "APPROVED" | "REJECTED" | "INVITED";
  createdAt: string;

  firstName: string;
  lastName: string;
  jobTitle: string | null;
  professionalEmail: string;
  phone: string | null;

  companyName: string;
  legalName: string | null;
  taxId: string;
  establishmentType: string;
  addressLine1: string | null;
  city: string | null;
  province: string | null;
  country: string | null;
  roomCount: number | null;
  website: string | null;

  pmsName: string | null;
  estimatedUsers: number | null;
  requestedModules: string[] | null;
  notes: string | null;
}
```

```
termsAccepted: boolean;
professionalUseAccepted: boolean;
termsAcceptedAt: string | null;

pdfStoragePath: string | null;
pdfUrl: string | null;

reviewedBy: string | null;
reviewedAt: string | null;
reviewNotes: string | null;
invitedAt: string | null;
}
```

10. Integración con el backend actual

10.1. Principio

No asumir nada hasta ver los ficheros reales.

Porque aquí hay varios puntos sensibles:

- no sabemos el payload exacto actual
- no sabemos si la Edge Function hace validaciones estrictas
- no sabemos si el PDF se genera en frontend o backend
- no sabemos si el admin consume una tabla o varias
- no sabemos si el flujo de invitación ya depende de ciertos nombres de campos

10.2. Qué debe comprobarse antes de tocar la función

1. Componente frontend actual de solicitud.
 2. Servicio o fetch que envía la solicitud.
 3. Edge Function actual.
 4. SQL de la tabla donde se guarda la solicitud.
 5. SQL o estructura del admin que revisa.
 6. Flujo que crea la invitación.
 7. Cualquier tipo TS existente para `AccessRequest`.
-

11. Integración del footer legal actual en la nueva capa pública

11.1. Decisión

Sí, debe mantenerse.

11.2. Motivo

Ya resuelve:

- documentación legal
- modal bien estructurado
- navegación por documentos
- contenido listo en producción

Rehacer eso ahora no aporta valor proporcional y sí añade riesgo.

11.3. Trabajo real necesario

Solo hay que:

- importarlo en la nueva shell pública
- revisar que el contraste visual encaje
- revisar overlay/z-index con el nuevo diseño
- revisar comportamiento responsive

11.4. Riesgo concreto a revisar

En tus capturas se ve que el modal legal ocupa una capa importante sobre la pantalla. Hay que revisar:

- z-index
 - bloqueo de scroll
 - foco / accesibilidad
 - comportamiento sobre navbar fija
-

12. Plan de implementación recomendado

Sprint 1 — Shell pública nueva

Objetivo

Tener la web pública nueva funcionando con rutas reales.

Tareas

- crear `LandingPage.tsx`
- crear `PublicNavbar.tsx`
- crear `PricingPage.tsx`

- crear `ContactPage.tsx`
- integrar `LegalFooter`
- conectar navegación con `react-router-dom`

Sin tocar aún

- Edge Functions
- BD
- flujo de solicitud

Sprint 2 — Login nuevo sobre backend actual

Tareas

- rehacer `LoginPage`
- conectar con la lógica real
- revisar recuperación de contraseña
- revisar errores

Sprint 3 — Solicitud de acceso nueva + análisis backend

Tareas

- mapear función actual
- diseñar payload nuevo
- adaptar tabla BD
- adaptar Edge Function
- adaptar PDF
- adaptar admin si hace falta

Sprint 4 — End-to-end real

Tareas

- probar solicitud completa
- comprobar PDF
- comprobar inserción BD
- comprobar revisión admin
- comprobar invitación
- comprobar creación de contraseña

Sprint 5 — Limpieza y hardening

Tareas

- eliminar componentes obsoletos
 - limpiar estilos
 - ajustar responsive
 - revisar errores de consola
 - revisar carga de imágenes
 - revisar UX legal
-

13. Validaciones y criterios de aceptación

13.1. Landing

- navegación pública sin errores
- todas las páginas cargan
- footer legal funciona
- header fijo no rompe modales

13.2. Login

- login correcto con credenciales válidas
- error correcto con credenciales inválidas
- enlace de recuperación operativo si existe

13.3. Solicitud de acceso

- todos los nuevos campos validan
 - no se envía sin aceptar condiciones
 - se genera PDF
 - se guarda PDF
 - se inserta solicitud completa
 - el admin puede revisarla
 - se puede lanzar invitación
-

14. Ficheros actuales que sí necesito ver

Sí. Para hacer esta migración bien, necesito que me pases los ficheros reales de producción relacionados con las conexiones y los campos actuales.

Prioridad alta

1. Archivo donde defines las rutas públicas (`App.tsx`, router principal o equivalente).
2. Componente actual de `login`.
3. Componente actual de `solicitar-acceso`.
4. Servicio o función frontend que envía la solicitud de acceso.
5. Edge Function que recibe la solicitud.
6. SQL o estructura de la tabla donde guardas la solicitud.
7. Código donde generas el PDF de aceptación.
8. Código del admin que revisa solicitudes o al menos la parte donde se leen.
9. Flujo que invita al usuario a crear contraseña.

Prioridad media

10. `LegalFooter` y componentes que abre.
 11. Tipos TS actuales relacionados con login y access request.
 12. Cualquier helper de validación actual.
 13. Configuración de storage para PDF si existe.
-

15. Orden práctico recomendado para seguir

Paso siguiente recomendado

Primero revisar:

1. router público actual
2. login actual
3. solicitud actual
4. edge function actual
5. tabla actual

Con eso ya se puede hacer un plan de migración exacto y no teórico.

Después

- te dejo la nueva `LandingPage.tsx` ya encajada con tus rutas reales

- luego `LoginPage` nueva conectada a tu backend real
 - luego `AccessRequestPage` nueva con payload ampliado y plan de cambios de BD / Edge Function
-

16. Conclusión ejecutiva

La migración es viable y tiene sentido.

La decisión correcta es:

- **mantener el backend y legal actual como base**
- **migrar la capa pública al diseño nuevo**
- **rehacer login y access request visualmente**
- **ampliar solicitud de acceso con cambios controlados en backend**

La parte más delicada no es la landing; es la solicitud de acceso y su flujo legal / admin.

Por tanto, sí: necesito que me pases los ficheros actuales de producción donde se vean exactamente las conexiones, payloads y campos actuales para darte la migración ya aterrizada sobre tu código real, sin suposiciones.

17. Hallazgos reales a partir del fichero actual `SolicitarAccesoPage.tsx`

Ya con este fichero se puede aterrizar bastante mejor la migración.

17.1. Qué hace hoy realmente la pantalla actual

La pantalla actual de solicitud de acceso ya no es un formulario tonto. Tiene un flujo bastante concreto:

1. Captura datos de contacto y empresa.
2. Crea una solicitud preliminar mediante `createDebacuEvalAccessRequestDraft(...)`.
3. Obliga a abrir el diálogo legal `LegalDialog`.
4. Tras aceptar, genera justificante PDF con `generateTermsAcceptancePdf(...)`.
5. Marca `accepted_terms = true` solo si el PDF se ha generado correctamente.
6. Finalmente envía / cierra la solicitud con `finalizeDebacuEvalAccessRequest(...)`.
7. Redirige a `/solicitud-enviada`.

Esto confirma que la lógica legal y operativa está bastante bien pensada y no conviene romperla.

17.2. Servicios backend detectados

Desde esta pantalla ya se identifican tres servicios clave:

- `createDebacuEvalAccessRequestDraft`
- `finalizeDebacuEvalAccessRequest`
- `generateTermsAcceptancePdf`

Y además un componente legal central:

- `LegalDialog`

Esto significa que la migración de `Solicitar acceso` no debe rehacerse desde cero, sino rediseñarse por encima de este flujo real.

17.3. Contrato actual de datos detectado

Campos hoy usados en frontend:

```
company_name
legal_name
cif
address
city
country
property_type
rooms_count
website
contact_name
contact_role
email
phone
accepted_terms
accepted_professional_use
notes
```

Observación importante

Ya estás pidiendo bastante más de lo que parecía inicialmente. No es un formulario mínimo. El salto a la nueva versión no parte de cero; parte de un contrato ya existente.

17.4. Validaciones actuales detectadas

Para abrir términos (`canOpenTerms`)

Se exige como mínimo:

- `company_name`
- `cif`
- `contact_name`
- `email`

Para enviar solicitud (`canSubmit`)

Se exige:

- `requestId`
- `accepted_terms === true`
- `accepted_professional_use === true`
- `company_name`
- `cif`
- `contact_name`
- `email`

Conclusión

La aceptación legal ya está correctamente encadenada con el PDF. Eso no debe simplificarse en la nueva versión.

17.5. Flujo legal actual confirmado

La aceptación de términos funciona así:

1. Se asegura que existe `requestId`.
2. Se abre `LegalDialog`.
3. Al aceptar dentro del diálogo, se llama a:

```
generateTermsAcceptancePdf({  
  request_id: id,  
  email: form.email,  
});
```

4. Si eso devuelve `proof`, entonces:
 - se guarda en `acceptanceProof`
 - se marca `accepted_terms: true`
 - se cierra el diálogo
 - se muestra toast de éxito

Conclusión clave

No basta con copiar el diseño nuevo. La pantalla nueva debe respetar este orden funcional exacto.

17.6. Qué partes del diseño nuevo se pueden aplicar sin problema

Se pueden migrar sin riesgo alto:

- layout de dos columnas

- panel visual izquierdo
- jerarquía visual del formulario
- estilo SaaS del encabezado y campos
- copy más profesional
- mejor responsive
- mejor UX de validación
- mejor CTA final

17.7. Qué partes del diseño nuevo requieren adaptación real

Estas no se pueden meter a ciegas:

- separar nombre y apellidos si el backend solo usa `contact_name`
 - añadir provincia si hoy no existe en payload
 - añadir PMS, nº usuarios o módulos si el backend no los guarda
 - ampliar PDF con nuevos campos
 - ampliar tabla BD
 - adaptar admin a nuevos campos
-

18. Implicaciones técnicas inmediatas de este fichero

18.1. `contact_name` hoy es un campo único

En el nuevo diseño mostraste nombre y apellidos por separado, pero hoy el sistema guarda:
`contact_name`

Decisión pendiente

Hay que decidir si:

Opción A

Se mantiene un solo campo lógico en backend y en UI se muestra un único campo “Nombre y apellidos”.

Opción B

Se separa realmente en:

- `first_name`
- `last_name`

Y entonces habrá que tocar:

- frontend
- Edge Function
- tabla BD
- admin
- PDF

Recomendación práctica

Para no romper nada en la primera migración, mantener de momento `contact_name` como campo único visual o componerlo en frontend y mandarlo unido al backend.

18.2. rooms_count hoy entra como string y se normaliza a número

Esto ya está controlado en frontend con `roomsCountNumber`.

No hay problema en mantenerlo, pero conviene validar mejor en la nueva versión:

- solo enteros positivos
- evitar texto libre

18.3. country hoy es texto simple

No parece haber selector normalizado de país. En la nueva versión puedes mantenerlo simple o mejorarlo a selector, pero si lo cambias a catálogo habrá que ver si backend espera un código ISO o texto libre.

18.4. property_type sí está tipado

Ya existe `PropertyType`, así que la nueva UI debe seguir usando ese tipo y no inventar otros valores.

18.5. accepted_terms no debe marcarse manualmente

Hoy ese campo solo debe activarse después de `generateTermsAcceptancePdf(...)`.

Esto es correcto y debe mantenerse así.

19. Decisiones recomendadas para la nueva SolicitarAccesoPage

19.1. Primera versión migrada recomendada

La primera versión nueva debe:

- mantener exactamente el backend actual
- mantener exactamente los mismos campos efectivos
- mejorar solo diseño y UX
- mantener LegalDialog
- mantener generación PDF
- mantener finalizeDebacuEvalAccessRequest

Es decir

Primero rediseño visual sin ampliar contrato de datos.

Eso reduce muchísimo el riesgo.

19.2. Segunda versión posterior

Después, si quieres más datos, entonces sí:

- ampliar payload
- ampliar tabla
- ampliar Edge Function
- ampliar PDF
- ampliar revisión admin

Esa segunda versión ya sería una evolución funcional, no solo una migración visual.

20. Qué ficheros necesito ahora, después de ver este

Con este fichero ya se puede avanzar bastante, pero todavía faltan piezas críticas.

Prioridad máxima ahora

1. `src/services/debacu_eval_accessRequests.service.ts`
 - clave total
 - aquí veremos payload real, endpoints y estructura de respuesta
2. `src/pages/legal/LegalDialog.tsx`
 - para entender cómo se integra la aceptación y si hay dependencias especiales
3. Edge Function o backend que ejecuta:
 - draft creation
 - finalize request

- generate pdf
4. SQL o estructura de la tabla donde se guarda la solicitud

Muy recomendable también

5. Pantalla o lógica admin que revisa solicitudes
 6. Flujo que envía invitación para crear contraseña
 7. LoginPage actual real en producción
 8. archivo de rutas públicas (App.tsx o router principal)
-

21. Recomendación operativa inmediata

A partir de lo visto, el orden más sensato es este:

Paso 1

Rehacer LandingPage y páginas públicas con diseño nuevo.

Paso 2

Rehacer SolicitarAccesoPage solo a nivel visual, manteniendo:

- mismo FormState efectivo
- mismos servicios
- mismo LegalDialog
- mismo flujo PDF

Paso 3

Rehacer LoginPage visualmente sobre su backend real.

Paso 4

Solo después decidir si amplías de verdad los campos del access request.

Ese orden reduce riesgo y te deja avanzar rápido sin romper la parte legal ni la lógica de aprobación.

SERVICIO:

```
// src/services/debacu_eval_accessRequests.service.ts
import { supabase } from "@services/supabaseClient";
```

```
export type PropertyType = "HOTEL" | "RURAL" | "APARTMENTS" | "HOSTEL" | "OTHER";
```

```
/**
```

```

* Draft input:
* - Recomendación: NO exigir accepted_professional_use aquí.
* - Se envía en finalize (flujo limpio).
*
* Nota: si tu Edge Function "create_draft" aún lo requiere, lo mandamos
* como false si no viene para mantener compatibilidad.
*/
export type DebaCuEvalAccessRequestDraftInput = {
  company_name: string;
  legal_name?: string;
  cif: string;
  address?: string;
  city?: string;
  country?: string;

  property_type: PropertyType;
  rooms_count?: number;

  website?: string;

  contact_name: string;
  contact_role?: string;
  email: string;
  phone?: string;

  accepted_professional_use?: boolean; // ☒ ahora opcional
  notes?: string;
};

/**
 * (Info) Mantén esto si lo usas en el front. La Edge Function de PDF ya fija versión server-side.
 */
export const TERMS_VERSION = "2026-01-24 - V1.0";

/**
 * Ojo: ahora tu Edge Function de PDF devuelve:
 * { proof: { request_id, bucket, path, sha256, accepted_at, terms_version } }
 * (sin signed_url)
 */
export type AcceptanceProof = {
  request_id?: string;

  bucket?: string;
  path?: string;
  pdf_path?: string; // por compat si algún día devolvías pdf_path

  signed_url?: string | null; // por compat (ya no se usa)

  sha256?: string;
  accepted_at?: string;
  terms_version?: string;

  ip?: string | null;

```



```

    user_agent?: string | null;

    already_accepted?: boolean;
  };

function assertOk<T>(data: any, fallbackMsg: string): T {
  if (!data) throw new Error(fallbackMsg);
  return data as T;
}

function normalizeEmail(e: string) {
  return String(e ?? "").trim().toLowerCase();
}

/**
 * Crea solicitud (legacy name "draft") pero realmente crea PENDING en BD
 */
export async function createDebacuEvalAccessRequestDraft(
  input: DebacuEvalAccessRequestDraftInput
): Promise<{ id: string; duplicate?: boolean }> {
  // ☒ compat: si backend aún exige accepted_professional_use, mandamos false por defecto
  const body = {
    ...input,
    email: normalizeEmail(input.email),
    accepted_professional_use: !!input.accepted_professional_use,
  };

  const { data, error } = await supabase.functions.invoke(
    "debacu_eval_access_request_create_draft",
    { body }
  );

  if (error) throw error;

  const out = assertOk<{ id: string; duplicate?: boolean }>(
    data,
    "Respuesta inválida del servidor (sin data)"
  );

  if (!out.id) throw new Error("Respuesta inválida del servidor (sin id)");
  return out;
}

/**
 * Genera PDF justificante (lee de BD, sube a Storage privado y actualiza la fila)
 *
 * ☒ Importante: pasa email para que el server valide que quien acepta
 * coincide con el email de la solicitud (evita modo PENDING<24h).
 */
export async function generateTermsAcceptancePdf(input: {
  request_id: string;
  email?: string;
}): Promise<{ proof: AcceptanceProof }> {
  const body = {

```

```

    request_id: input.request_id,
    email: input.email ? normalizeEmail(input.email) : undefined,
  };

  const { data, error } = await supabase.functions.invoke(
    "debacu_eval_generate_terms_acceptance",
    { body }
  );

  if (error) throw error;

  const out = assertOk<{ proof: AcceptanceProof }>(
    data,
    "Respuesta inválida del servidor (sin proof)"
  );

  if (!out.proof?.request_id) {
    throw new Error("Respuesta inválida del servidor (proof incompleto)");
  }

  return out;
}

/**
 * Finaliza (legacy): en el modelo nuevo SOLO valida que:
 * - existe request_id
 * - accepted_terms = true
 * - accepted_terms_pdf_path existe
 * y actualiza accepted_professional_use
 *
 * [✓] Tu Edge Function finalize espera:
 * { request_id: string; accepted_professional_use: boolean }
 */
export async function finalizeDebacuEvalAccessRequest(input: {
  request_id: string;
  accepted_professional_use: boolean;
}): Promise<{ ok: true }> {
  const body = {
    request_id: input.request_id,
    accepted_professional_use: !!input.accepted_professional_use,
  };

  const { data, error } = await supabase.functions.invoke(
    "debacu_eval_access_request_finalize",
    { body }
  );

  if (error) throw error;

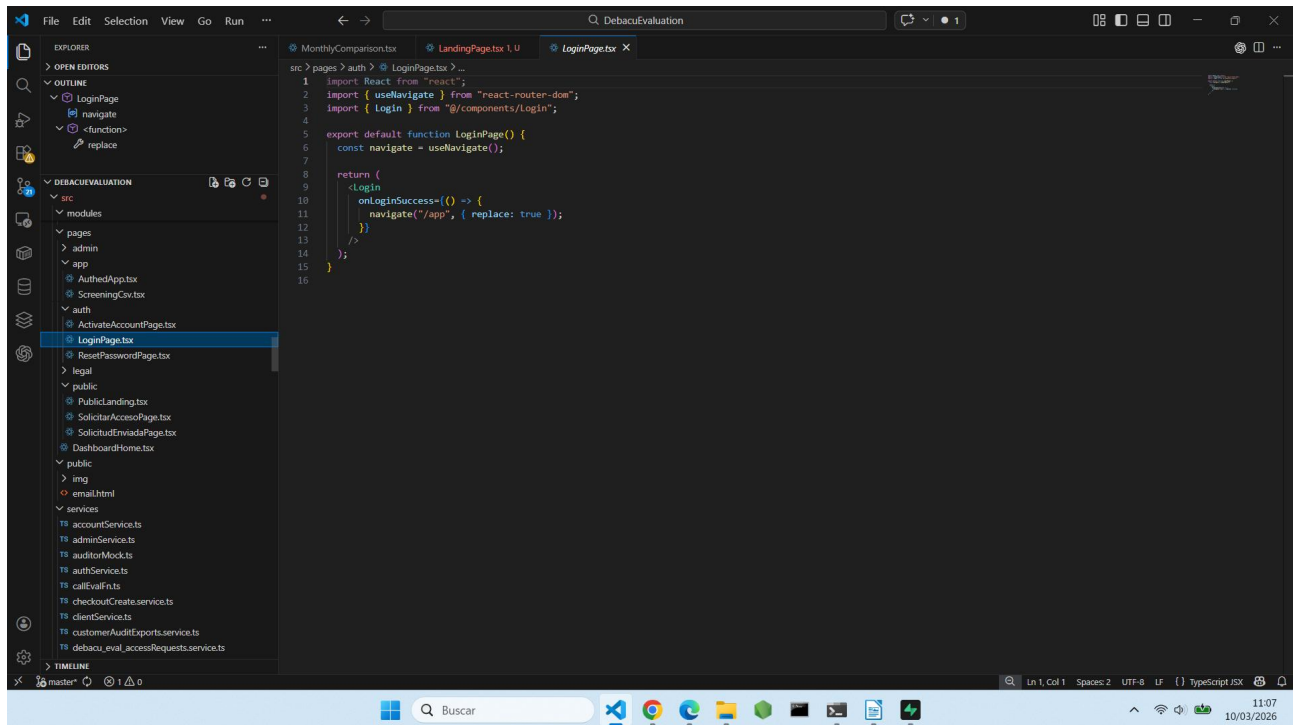
  const out = assertOk<{ ok: true }>(
    data,
    "Respuesta inválida del servidor (sin ok)"
  );
}

```

```

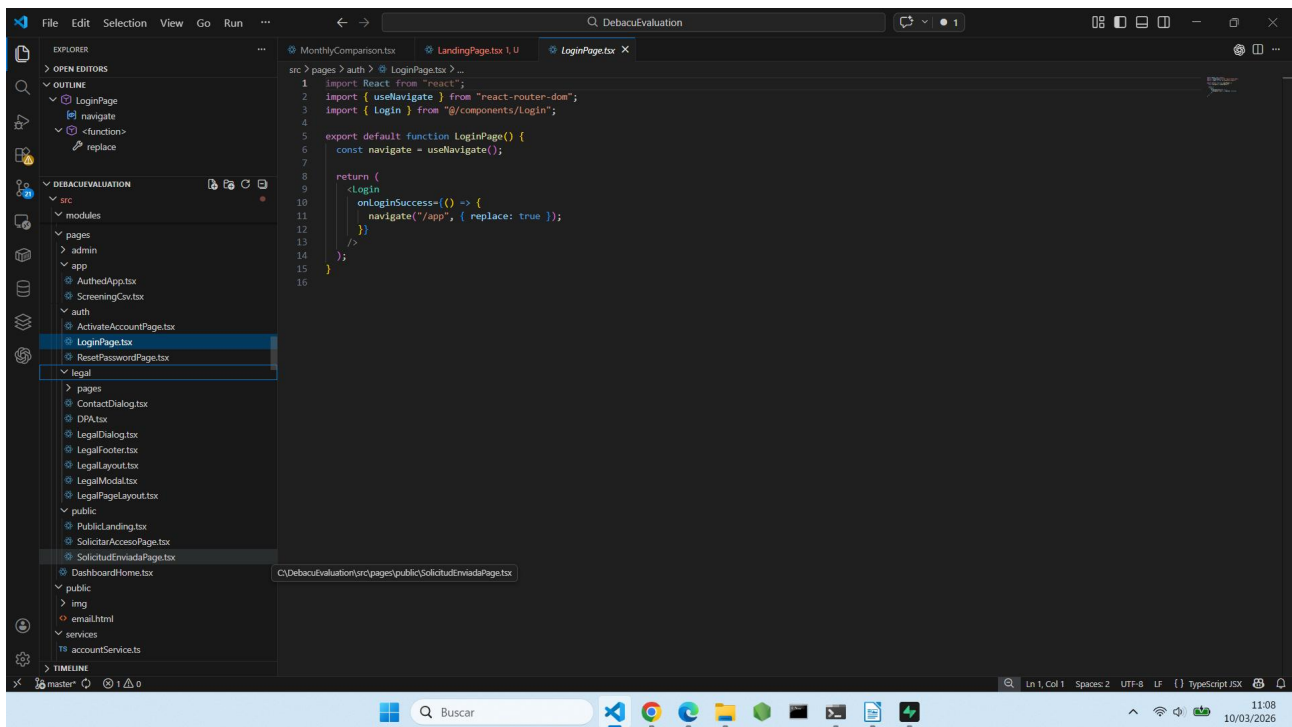
if (!out.ok) throw new Error("No se pudo finalizar la solicitud");
return out;
}

```

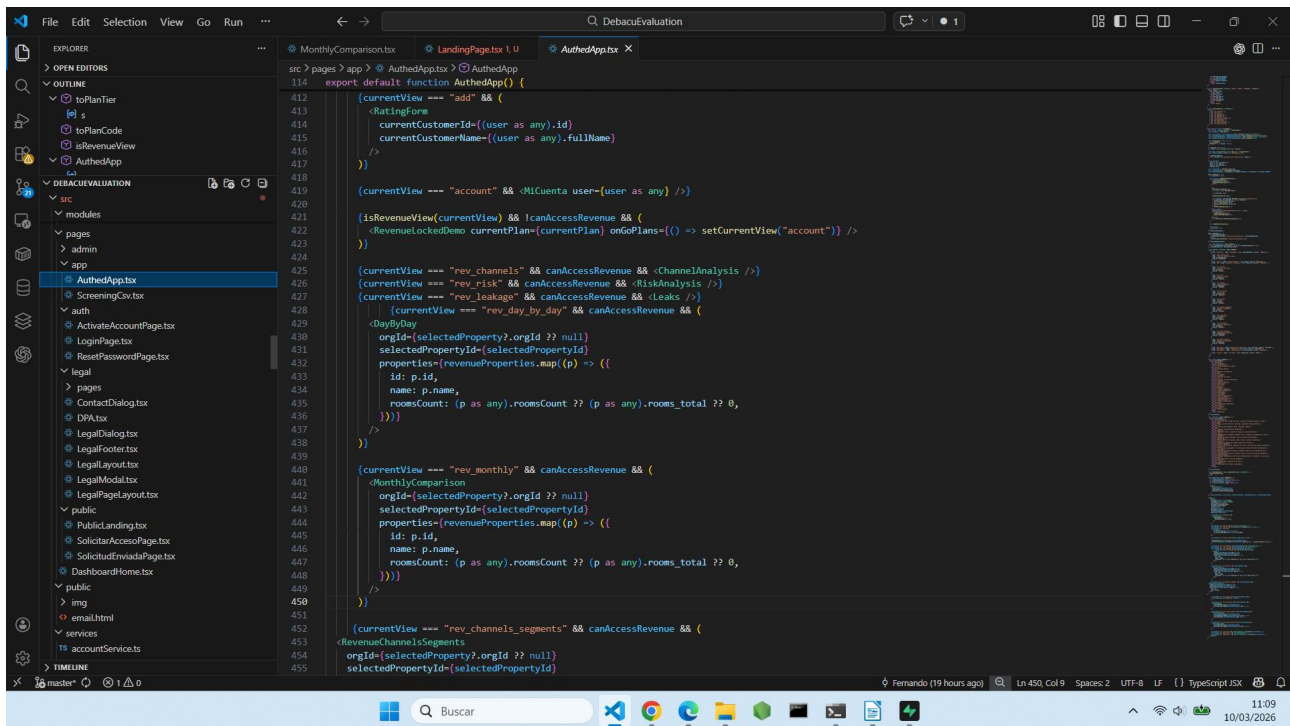


DONDE ESTA LOGIN ACTUAL (iMAGEN)

ARBOL DE CARPETAS



ROUTERS:



DENTRO DE src/pages/app/AuthedApp.tsx

actual: `import React from "react";`
`import { Navigate, useNavigate } from "react-router-dom";`

```
import { SearchRatings } from "@components/SearchRatings";
import { RatingForm } from "@components/RatingForm";
import { MiCuenta } from "@components/account/MiCuenta";
import AppShell, { type AuthedView, type NavItem } from "@components/layout/AppShell";
import DashboardHome from "@pages/DashboardHome";
import { useEvalAuth } from "@context/EvalAuthContext";
```

```
// Revenue Intelligence
import ChannelAnalysis from "@views/ChannelAnalysis";
import RiskAnalysis from "@views/RiskAnalysis";
import Leaks from "@views/Leaks";
import DayByDay from "@modules/revenue-intelligence/pages/DayByDay";
import MonthlyComparison from "@modules/revenue-intelligence/pages/MonthlyComparison";
import RevenueChannelsSegments from "@modules/revenue-intelligence/pages/revenueChannelsSegments";
import SettingsProperties from "@modules/revenue-intelligence/pages/SettingsProperties";
import RoomTypesPage from "@modules/revenue-intelligence/pages/RoomTypesPage";
import PriceCalendarPage from "@modules/revenue-intelligence/pages/PriceCalendarPage";
import EventsSeasonsPage from "@modules/revenue-intelligence/pages/EventsSeasonsPage";
import PropertySelector from "@modules/revenue-intelligence/components/PropertySelector";
import {
  getProperties,
  type RevenueProperty,
} from "@modules/revenue-intelligence/services/revenueProperties.service";
```

```
// Demo/PAYWALL Revenue
```

```
import RevenueLockedDemo from "@components/revenue/RevenueLockedDemo";
```

```
// Auditoria
```

```
import StatsViewAuditor from "@views/StatsViewAuditor";
```

```
import HistoryViewAuditor from "@views/HistoryViewAuditor";
```

```
import ExportsViewAuditor from "@views/ExportsViewAuditor";
```

```
// Screening CSV
```

```
import ScreeningCsv from "@pages/app/ScreeningCsv";
```

```
import {
```

```
  LayoutDashboard,
```

```
  Search,
```

```
  PlusCircle,
```

```
  BarChart3,
```

```
  ShieldAlert,
```

```
  TrendingDown,
```

```
  Activity,
```

```
  Clock,
```

```
  Download,
```

```
  CreditCard,
```

```
  FileSpreadsheet,
```

```
  Building2,
```

```
  BedDouble,
```

```
  CalendarRange,
```

```
  CalendarDays,
```

```
  LineChart,
```

```
  CalendarClock,
```

```
  Layers3,
```

```
} from "lucide-react";
```

```
import { PlanType } from "@types/types";
```

```
import { PlanTier } from "../../auditor";
```

```
function toPlanTier(planLike: any): PlanTier {
```

```
  const s = String(planLike ?? "").toUpperCase();
```

```
  if (s === "FREE") return PlanTier.FREE;
```

```
  if (s === "BASIC") return PlanTier.BASIC;
```

```
  if (s === "MEDIUM") return PlanTier.MEDIUM;
```

```
  if (s === "PREMIUM") return PlanTier.PREMIUM;
```

```
  switch (planLike as PlanType) {
```

```
    case PlanType.INACTIVE:
```

```
      return PlanTier.FREE;
```

```
    case PlanType.BASIC:
```

```
      return PlanTier.BASIC;
```

```
    case PlanType.PROFESSIONAL:
```

```
      return PlanTier.MEDIUM;
```

```
    case PlanType.ENTERPRISE:
```

```
      return PlanTier.PREMIUM;
```

```
    default:
```

```
      return PlanTier.BASIC;
```

```
  }
```

```
}
```

```

function toPlanCode(plan: PlanTier): "FREE" | "BASIC" | "MEDIUM" | "PREMIUM" {
  switch (plan) {
    case PlanTier.FREE:
      return "FREE";
    case PlanTier.BASIC:
      return "BASIC";
    case PlanTier.MEDIUM:
      return "MEDIUM";
    case PlanTier.PREMIUM:
      return "PREMIUM";
    default:
      return "BASIC";
  }
}

```

```

function isRevenueView(v: AuthedView) {
  return (
    v === "rev_channels" ||
    v === "rev_risk" ||
    v === "rev_leakage" ||
    v === "rev_day_by_day" ||
    v === "rev_monthly" ||
    v === "rev_channels_segments" ||
    v === "rev_properties" ||
    v === "rev_room_types" ||
    v === "rev_price_calendar" ||
    v === "rev_events_seasons"
  );
}

```

```

export default function AuthedApp() {
  const { user, loading, signOut } = useEvalAuth();
  const navigate = useNavigate();

  const [currentView, setCurrentView] = React.useState<AuthedView>("dashboard");
  const [revenueProperties, setRevenueProperties] = React.useState<RevenueProperty[]>([]);
  const [selectedPropertyId, setSelectedPropertyId] = React.useState<string | null>(null);
  const [propertiesLoading, setPropertiesLoading] = React.useState(false);

  const handleLogout = async () => {
    await signOut();
    navigate("/", { replace: true });
  };

  if (loading) return null;
  if (!user) return <Navigate to="/login" replace />;

  const email = String((user as any).email ?? "").toLowerCase();
  const isPlatformAdmin = email === "admin@debacu.com";

  if (isPlatformAdmin) {
    return <Navigate to="/app/admin/solicitudes-acceso" replace />;
  }
}

```

```

const planLike =
  (user as any).currentPlan ??
  (user as any).plan ??
  (user as any).planType ??
  PlanType.BASIC;

const currentPlan = toPlanTier(planLike);
const currentPlanCode = toPlanCode(currentPlan);
const canAccessRevenue = currentPlan === PlanTier.MEDIUM || currentPlan === PlanTier.PREMIUM;

```

```

React.useEffect(() => {
  let cancelled = false;

```

```

  async function loadRevenueProperties() {
    if (!canAccessRevenue) {
      setRevenueProperties([]);
      setSelectedPropertyId(null);
      return;
    }

```

```

    try {
      setPropertiesLoading(true);
      const rows = await getProperties();

```

```

    if (cancelled) return;

```

```

    setRevenueProperties(rows);

```

```

    const savedId = localStorage.getItem("revenue_active_property_id");
    if (savedId && rows.some((p) => p.id === savedId)) {
      setSelectedPropertyId(savedId);
    } else if (rows.length > 0) {
      setSelectedPropertyId(rows[0].id);
    } else {
      setSelectedPropertyId(null);
    }
  } catch (error) {
    console.error("loadRevenueProperties error:", error);
    if (!cancelled) {
      setRevenueProperties([]);
      setSelectedPropertyId(null);
    }
  } finally {
    if (!cancelled) setPropertiesLoading(false);
  }
}

```

```

void loadRevenueProperties();

```

```

return () => {
  cancelled = true;

```

```
};  
}, [canAccessRevenue]);
```

```
React.useEffect(() => {  
  if (selectedPropertyId) {  
    localStorage.setItem("revenue_active_property_id", selectedPropertyId);  
  } else {  
    localStorage.removeItem("revenue_active_property_id");  
  }  
}, [selectedPropertyId]);
```

```
const selectedProperty = React.useMemo(() => {  
  return revenueProperties.find((p) => p.id === selectedPropertyId) ?? null;  
}, [revenueProperties, selectedPropertyId]);
```

```
const navItems: NavItem[] = React.useMemo(  
  () => [  
    { view: "dashboard", label: "Dashboard", icon: LayoutDashboard, section: "OPERATIVA" },  
    {  
      view: "aud_screening_csv",  
      label: "Consulta automática (CSV)",  
      icon: FileSpreadsheet,  
      section: "OPERATIVA",  
    },  
    { view: "search", label: "Consulta manual", icon: Search, section: "OPERATIVA" },  
    { view: "add", label: "Registrar incidencia", icon: PlusCircle, section: "OPERATIVA" },
```

```
{  
  view: "rev_channels",  
  label: "Análisis por Canal",  
  icon: BarChart3,  
  section: "REVENUE",  
},  
{  
  view: "rev_risk",  
  label: "Nivel de Riesgo",  
  icon: ShieldAlert,  
  section: "REVENUE",  
},  
{  
  view: "rev_leakage",  
  label: "Fugas de Revenue",  
  icon: TrendingDown,  
  section: "REVENUE",  
},  
{  
  view: "rev_day_by_day",  
  label: "Día x Día",  
  icon: CalendarClock,  
  section: "REVENUE",  
},  
{
```



```

    view: "rev_monthly",
    label: "Mensual",
    icon: LineChart,
    section: "REVENUE",
  },
  {
    view: "rev_channels_segments",
    label: "Canales & Segmentos",
    icon: Layers3,
    section: "REVENUE",
  },
  {
    view: "rev_properties",
    label: "Propiedades",
    icon: Building2,
    section: "REVENUE",
  },
  {
    view: "rev_room_types",
    label: "Tipos de habitación",
    icon: BedDouble,
    section: "REVENUE",
  },
  {
    view: "rev_price_calendar",
    label: "Calendario de precios",
    icon: CalendarRange,
    section: "REVENUE",
  },
  {
    view: "rev_events_seasons",
    label: "Eventos y temporadas",
    icon: CalendarDays,
    section: "REVENUE",
  },
},

```

```

{ view: "aud_stats", label: "Estadísticas operativas", icon: Activity, section: "AUDITORIA" },
{ view: "aud_history", label: "Histórico", icon: Clock, section: "AUDITORIA" },
{ view: "aud_exports", label: "Exportaciones", icon: Download, section: "AUDITORIA" },

```

```

{ view: "account", label: "Mi cuenta", icon: CreditCard, section: "CUENTA" },
],
[]
);

```

```

const title = React.useMemo(() => {
  switch (currentView) {
    case "dashboard":
      return "Dashboard";
    case "aud_screening_csv":
      return "Consulta automática (CSV)";
    case "search":

```

```

        return "Consulta manual";
    case "add":
        return "Registrar incidencia";
    case "account":
        return "Mi cuenta";
    case "rev_channels":
        return "Análisis por Canal";
    case "rev_risk":
        return "Análisis por Nivel de Riesgo";
    case "rev_leakage":
        return "Fugas de Revenue";
    case "rev_day_by_day":
        return "Día x Día";
    case "rev_monthly":
        return "Mensual";
    case "rev_channels_segments":
        return "Canales & Segmentos";
    case "rev_properties":
        return "Propiedades";
    case "rev_room_types":
        return "Tipos de habitación";
    case "rev_price_calendar":
        return "Calendario de precios";
    case "rev_events_seasons":
        return "Eventos y temporadas";
    case "aud_stats":
        return "Estadísticas operativas";
    case "aud_history":
        return "Histórico";
    case "aud_exports":
        return "Exportaciones";
    default:
        return "Dashboard";
    }
  }, [currentView]);

```

```

const subtitle = React.useMemo(() => {
  switch (currentView) {
    case "aud_screening_csv":
      return "Importa un CSV, valida (dry-run) y ejecuta screening (persona + fecha).";
    case "account":
      return "Planes, perfil del hotel, catálogo, seguridad y datos bancarios.";
    case "search":
      return "Consulta por documento, email, teléfono o nombre.";
    case "add":
      return "Registro estructurado de incidencias.";
    case "rev_channels":
      return "Identifica cómo se reparte el riesgo por canal/plataforma.";
    case "rev_risk":
      return "Segmentación del impacto económico real y volumen de incidencias por nivel.";
    case "rev_leakage":
      return "Ranking de fugas de margen y desvíos sobre precio esperado.";
  }

```

```

    case "rev_day_by_day":
      return "Detalle diario de ocupación, ADR, revenue y eventos operativos.";
    case "rev_monthly":
      return "Comparativa mensual de revenue, ADR, RN y evolución.";
    case "rev_channels_segments":
      return "Análisis de producción comercial por canal y estructura de ventas disponible.";
    case "rev_properties":
      return "Gestiona las propiedades y la configuración base de Revenue Intelligence.";
    case "rev_room_types":
      return "Gestiona el inventario base por categoría dentro de la propiedad activa.";
    case "rev_price_calendar":
      return "Configura precio diario, estancia mínima y cierres por tipo de habitación.";
    case "rev_events_seasons":
      return "Gestiona temporadas y eventos que contextualizan el calendario y el pricing.";
    case "aud_stats":
      return "KPIs operativos y métricas agregadas.";
    case "aud_history":
      return "Trazabilidad y registro de acciones.";
    case "aud_exports":
      return "Exportación de informes y descargas.";
    default:
      return "";
  }
}, [currentView]);

```

```

const handleNavigate = React.useCallback((view: AuthedView) => {
  setCurrentView(view);
}, []);

```

```

const headerLeft = React.useMemo(() => {
  if (!canAccessRevenue) return null;
  if (!isRevenueView(currentView)) return null;
  if (propertiesLoading) return null;
  if (!revenueProperties.length) return null;

```

```

  return (
    <PropertySelector
      properties={revenueProperties}
      selectedId={selectedPropertyId}
      onSelect={setSelectedPropertyId}
    />
  );
}, [canAccessRevenue, currentView, propertiesLoading, revenueProperties, selectedPropertyId]);

```

```

return (
  <AppShell
    userEmail={{user as any}.email}
    userName={{user as any}.fullName}
    activeView={currentView}
    onNavigate={handleNavigate}
    onLogout={handleLogout}
    title={title}

```

```

        subtitle={subtitle}
        navItems={navItems}
        currentPlanCode={currentPlanCode}
        headerLeft={headerLeft}
    >
    {currentView === "dashboard" && (
        <DashboardHome
            onNavigate={(v) => {
                handleNavigate(v as any);
            }}
        />
    )}

```

```

    {currentView === "aud_screening_csv" && <ScreeningCsv />}
    {currentView === "search" && <SearchRatings currentUser={user as any} />}
    {currentView === "add" && (
        <RatingForm
            currentCustomerId={(user as any).id}
            currentCustomerName={(user as any).fullName}
        />
    )}

```

```

    {currentView === "account" && <MiCuenta user={user as any} />}

```

```

    {isRevenueView(currentView) && !canAccessRevenue && (
        <RevenueLockedDemo currentPlan={currentPlan} onGoPlans={() => setCurrentView("account")} />
    )}

```

```

    {currentView === "rev_channels" && canAccessRevenue && <ChannelAnalysis />}
    {currentView === "rev_risk" && canAccessRevenue && <RiskAnalysis />}
    {currentView === "rev_leakage" && canAccessRevenue && <Leaks />}
    {currentView === "rev_day_by_day" && canAccessRevenue && (
        <DayByDay
            orgId={selectedProperty?.orgId ?? null}
            selectedPropertyId={selectedPropertyId}
            properties={revenueProperties.map((p) => ({
                id: p.id,
                name: p.name,
                roomsCount: (p as any).roomsCount ?? (p as any).rooms_total ?? 0,
            })
        )}
        />
    )}

```

```

    {currentView === "rev_monthly" && canAccessRevenue && (
        <MonthlyComparison
            orgId={selectedProperty?.orgId ?? null}
            selectedPropertyId={selectedPropertyId}
            properties={revenueProperties.map((p) => ({
                id: p.id,
                name: p.name,
                roomsCount: (p as any).roomsCount ?? (p as any).rooms_total ?? 0,
            })
        )}
    )}

```

```

    />
  )}

  {currentView === "rev_channels_segments" && canAccessRevenue && (
    <RevenueChannelsSegments
      orgId={selectedProperty?.orgId ?? null}
      selectedPropertyId={selectedPropertyId}
      properties={revenueProperties.map((p) => ({
        id: p.id,
        name: p.name,
      })))}
    />
  )}

  {currentView === "rev_properties" && canAccessRevenue && (
    <SettingsProperties user={user as any} />
  )}

  {currentView === "rev_room_types" && canAccessRevenue && (
    <RoomTypesPage
      selectedPropertyId={selectedPropertyId}
      selectedPropertyName={selectedProperty?.name ?? null}
    />
  )}

  {currentView === "rev_price_calendar" && canAccessRevenue && (
    <PriceCalendarPage
      selectedPropertyId={selectedPropertyId}
      selectedPropertyName={selectedProperty?.name ?? null}
      selectedOrgId={selectedProperty?.orgId ?? null}
    />
  )}

  {currentView === "rev_events_seasons" && canAccessRevenue && (
    <EventsSeasonsPage
      selectedPropertyId={selectedPropertyId}
      selectedPropertyName={selectedProperty?.name ?? null}
    />
  )}

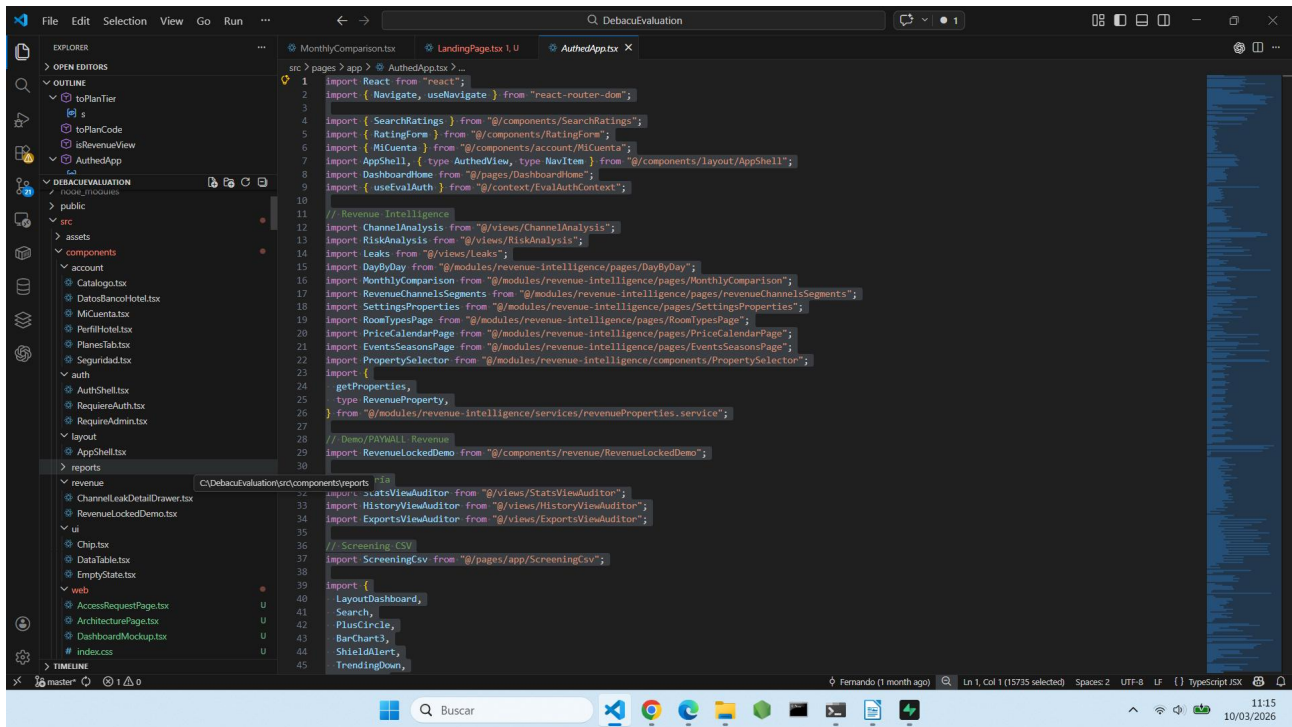
  {currentView === "aud_stats" && <StatsViewAuditor currentPlan={currentPlan} />}
  {currentView === "aud_history" && <HistoryViewAuditor />}
  {currentView === "aud_exports" && <ExportsViewAuditor currentPlan={currentPlan} />}
</AppShell>
);
}

```

en carpeta src/legal/**** documentos legales y tambien
 en carpeta src/legal/pages/***** todas lo de legal

unas son para la web y otras se cargan en el modal de aceptacion

en carpeta src/public/ publicLandig.tsx, SolicitarAccesoPage.tsx y SolicitudEnviadaPage.tsx



EN LA CARPETA SERC/COMPONENTS/LAYOUT/ AppShell.tsx `import React, { useEffect, useMemo, useState } from "react";`

```
import {  
  LayoutDashboard,  
  Search,  
  PlusCircle,  
  CreditCard,  
  Logout,  
  Menu,  
  BarChart3,  
  ShieldAlert,  
  TrendingDown,  
  FileText,  
  Shield,  
  Activity,  
  Clock,  
  Download,  
  Settings,  
  Lock,  
  Building2,  
  BedDouble,  
  CalendarRange,  
  CalendarDays,  
  LineChart,  
  CalendarClock,  
  Layers3,  
} from "lucide-react";
```

```
export type AuthedView =  
  | "dashboard"  
  | "search"
```

```

| "add"
| "account"
| "admin"
| "rev_channels"
| "rev_risk"
| "rev_leakage"
| "rev_day_by_day"
| "rev_monthly"
| "rev_channels_segments"
| "rev_properties"
| "rev_room_types"
| "rev_price_calendar"
| "rev_events_seasons"
| "aud_summary"
| "aud_risk"
| "aud_stats"
| "aud_history"
| "aud_exports"
| "aud_config"
| "aud_screening_csv";

```

```

export type NavItem = {
  view: AuthedView;
  label: string;
  icon: React.ComponentType<{ className?: string }>;
  disabled?: boolean;
  locked?: boolean;
  section?: "OPERATIVA" | "REVENUE" | "AUDITORIA" | "CUENTA";
};

```

```

type Props = {
  userEmail?: string;
  userName?: string;
  activeView: AuthedView;
  onNavigate: (view: AuthedView) => void;
  onLogout: () => void;
  title?: string;
  subtitle?: string;
  children: React.ReactNode;
  navItems?: NavItem[];
  showAccountActions?: boolean;
  currentPlanCode?: string;
  headerLeft?: React.ReactNode;
};

```

```

function classNames(...parts: Array<string | false | null | undefined>) {
  return parts.filter(Boolean).join(" ");
}

```

```

const SectionHeader: React.FC<{ children: React.ReactNode }> = ({ children }) => (
  <div className="px-4 pt-4 pb-2 text-[10px] font-medium text-slate-400 uppercase tracking-wide">
    {children}
  </div>
);

```

```

const NavItemButton: React.FC<{
  view: AuthedView;
  label: string;
  icon: React.ComponentType<{ className?: string }>;
  disabled?: boolean;
  locked?: boolean;
  active: boolean;
  onClick: () => void;
}> = React.memo(({ view, label, icon: Icon, disabled, locked, active, onClick }) => {
  const isLocked = !!locked && !disabled;

  return (
    <button
      type="button"
      disabled={disabled}
      onClick={onClick}
      className={classNames(
        "w-full flex items-center gap-3 px-4 py-2.5 rounded-lg text-sm transition-colors duration-150",
        "focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-blue-500/30",
        disabled
          ? "text-slate-400 border border-dashed border-slate-200 bg-slate-50 cursor-not-allowed"
          : active
            ? "bg-blue-50 text-blue-700 border border-blue-100 shadow-sm"
            : isLocked
              ? "text-slate-500 hover:bg-slate-50 border border-slate-100"
              : "text-slate-600 hover:bg-slate-50 hover:text-slate-900"
        )}
      aria-current={active ? "page" : undefined}
      data-view={view}
      title={isLocked ? "Disponible en MEDIUM / PREMIUM" : undefined}
    >
      <Icon
        className={classNames(
          "w-4 h-4",
          disabled
            ? "text-slate-300"
            : active
              ? "text-blue-600"
              : isLocked
                ? "text-slate-400"
                : "text-slate-400"
          )}
      />
      <span className="font-medium tracking-tight">{label}</span>
      {isLocked ? <Lock className="w-4 h-4 text-slate-400 ml-auto" /> : null}
    </button>
  );
});

NavItemButton.displayName = "NavItemButton";

const SectionBlock: React.FC<{
  title: React.ReactNode;

```



```

children: React.ReactNode;
}> = ({ title, children }) => (
  <div className="space-y-2">
    <SectionHeader>{title}</SectionHeader>
    <div className="space-y-1">{children}</div>
  </div>
);

```

```

export default function AppShell({
  userEmail,
  userName,
  activeView,
  onNavigate,
  onLogout,
  title,
  subtitle,
  children,
  navItems,
  showAccountActions = true,
  currentPlanCode,
  headerLeft,
}: Props) {
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);

```

```

const plan = (currentPlanCode ?? "").toUpperCase();
const canAccessRevenue = plan === "MEDIUM" || plan === "PREMIUM";

```

```

const nav = useMemo<NavItem[]>(() => {
  const base: NavItem[] = [
    { view: "dashboard", label: "Dashboard", icon: LayoutDashboard, section: "OPERATIVA" },
    { view: "search", label: "Consultar", icon: Search, section: "OPERATIVA" },
    { view: "add", label: "Registrar incidencia", icon: PlusCircle, section: "OPERATIVA" },

```

```

    {
      view: "rev_channels",
      label: "Análisis por Canal",
      icon: BarChart3,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_risk",
      label: "Nivel de Riesgo",
      icon: ShieldAlert,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_leakage",
      label: "Fugas de Revenue",
      icon: TrendingDown,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },

```

```

    },
    {
      view: "rev_day_by_day",
      label: "Día x Día",
      icon: CalendarClock,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_monthly",
      label: "Mensual",
      icon: LineChart,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_channels_segments",
      label: "Canales & Segmentos",
      icon: Layers3,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_properties",
      label: "Propiedades",
      icon: Building2,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_room_types",
      label: "Tipos de habitación",
      icon: BedDouble,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_price_calendar",
      label: "Calendario de precios",
      icon: CalendarRange,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
    {
      view: "rev_events_seasons",
      label: "Eventos y temporadas",
      icon: CalendarDays,
      section: "REVENUE",
      locked: !canAccessRevenue,
    },
  ],

```

```

  { view: "aud_screening_csv", label: "Screening CSV", icon: FileText, section: "AUDITORIA" },

```

```

    { view: "aud_summary", label: "Resumen", icon: FileText, section: "AUDITORIA" },
    { view: "aud_risk", label: "Auditoría de riesgo", icon: Shield, section: "AUDITORIA" },
    { view: "aud_stats", label: "Estadísticas operativas", icon: Activity, section: "AUDITORIA" },
    { view: "aud_history", label: "Histórico", icon: Clock, section: "AUDITORIA" },
    { view: "aud_exports", label: "Exportaciones", icon: Download, section: "AUDITORIA" },
    { view: "aud_config", label: "Configuración-Avisos", icon: Settings, section: "AUDITORIA" },

```

```

    { view: "account", label: "Mi cuenta", icon: CreditCard, section: "CUENTA" },
  ];

```

```

const src = navItems ?? base;

```

```

return src.map((i) => {
  if (i.section === "REVENUE") return { ...i, locked: !canAccessRevenue };
  return i;
});
}, [navItems, canAccessRevenue]);

```

```

const sections = useMemo(() => {
  const operativa = nav.filter((i) => (i.section ?? "OPERATIVA") === "OPERATIVA");
  const revenue = nav.filter((i) => i.section === "REVENUE");
  const auditoria = nav.filter((i) => i.section === "AUDITORIA");
  const cuenta = nav.filter((i) => i.section === "CUENTA");
  return { operativa, revenue, auditoria, cuenta };
}, [nav]);

```

```

useEffect(() => {
  if (!mobileMenuOpen) return;
  const onKeyDown = (e: KeyboardEvent) => {
    if (e.key === "Escape") setMobileMenuOpen(false);
  };
  window.addEventListener("keydown", onKeyDown);
  return () => window.removeEventListener("keydown", onKeyDown);
}, [mobileMenuOpen]);

```

```

const handleNavigate = (view: AuthedView) => {
  onNavigate(view);
  setMobileMenuOpen(false);
};

```

```

const renderItem = (items: NavItem[]) =>
  items.map((i) => (
    <NavItemButton
      key={i.view}
      view={i.view}
      label={i.label}
      icon={i.icon}
      disabled={i.disabled}
      locked={i.locked}
      active={activeView === i.view}
      onClick={() => {
        if (i.disabled) return;

```

```

        handleNavigate(i.view);
    }}
    />
  ));

  return (
    <div className="flex h-screen bg-slate-50">
      <aside className="hidden md:flex flex-col w-72 bg-white border-r border-slate-200">
        <div className="p-6 border-b border-slate-100">
          <div className="flex items-center gap-3">
            <div className="h-10 w-10 rounded-2xl bg-slate-900 text-white flex items-center justify-center font-bold">
              D
            </div>
            <div className="leading-tight">
              <div className="text-sm font-semibold text-slate-900">Debacu Evaluation360</div>
              <div className="text-xs text-slate-500">Uso profesional · Acceso restringido</div>
            </div>
          </div>
        </div>

        <div className="flex-1 overflow-y-auto p-4 pb-6 space-y-8">
          <SectionBlock title="Operativa">{renderItems(sections.operativa)}</SectionBlock>

          {sections.revenue.length > 0 && (
            <SectionBlock title="Revenue Intelligence">{renderItems(sections.revenue)}</SectionBlock>
          )}

          {sections.auditoria.length > 0 && (
            <SectionBlock title="Auditoría">{renderItems(sections.auditoria)}</SectionBlock>
          )}

          {showAccountActions && sections.cuenta.length > 0 && (
            <SectionBlock title="Cuenta">{renderItems(sections.cuenta)}</SectionBlock>
          )}
        </div>

        <div className="px-4 py-5 border-t border-slate-100 bg-slate-50">
          <div className="flex items-center gap-3 mb-3 px-2">
            <div className="w-9 h-9 rounded-2xl bg-slate-200 flex items-center justify-center text-slate-700 font-bold">
              {(userName?.[0] || userEmail?.[0] || "U").toUpperCase()}
            </div>
            <div className="overflow-hidden">
              <p className="text-sm font-semibold text-slate-900 truncate">{userName || "Usuario"}</p>
              <p className="text-xs text-slate-500 truncate">{userEmail || ""}</p>
            </div>
          </div>

          <button
            type="button"
            onClick={onLogout}

```

```

        className="w-full flex items-center justify-center gap-2 px-4 py-2 bg-white border border-slate-200 rounded-xl text-sm font-semibold text-slate-700 hover:bg-slate-50 transition-colors"
      >
        <Logout className="w-4 h-4" />
        Salir
      </button>
    </div>
  </aside>

```

```

<div className="flex-1 flex flex-col h-full overflow-hidden">
  <header className="md:hidden bg-white border-b border-slate-200 px-4 py-3 flex justify-between items-center">
    <button
      type="button"
      aria-label="Abrir menú"
      onClick={() => setMobileMenuOpen(true)}
      className="p-2 text-slate-700 rounded-lg hover:bg-slate-50"
    >
      <Menu className="w-6 h-6" />
    </button>

```

```

    <div className="text-sm font-semibold text-slate-900 truncate max-w-[55%]">
      {title || "Debacu Evaluation360"}
    </div>

```

```

    <div className="w-10" />
  </header>

```

```

    {mobileMenuOpen && (
      <div
        className="md:hidden fixed inset-0 z-50 bg-black/30"
        onClick={() => setMobileMenuOpen(false)}
        role="presentation"
      >
        <div
          className="absolute left-0 top-0 h-full w-[85%] max-w-sm bg-white border-r border-slate-200 px-4 py-4"
          onClick={(e) => e.stopPropagation()}
          role="dialog"
          aria-modal="true"
          aria-label="Menú de navegación"
        >
          <div className="flex items-center justify-between mb-4">
            <div className="text-sm font-semibold text-slate-900">Menú</div>
            <button
              type="button"
              aria-label="Cerrar menú"
              onClick={() => setMobileMenuOpen(false)}
              className="text-slate-600 px-2 py-1 rounded-lg hover:bg-slate-100"
            >
              ×
            </button>
          </div>

```

```

<div className="space-y-8 overflow-y-auto h-[calc(100%-56px)] pr-1">
  <SectionBlock title="Operativa">{renderItems(sections.operativa)}</SectionBlock>

```

```

    {sections.revenue.length > 0 && (
      <SectionBlock title="Revenue Intelligence">{renderItems(sections.revenue)}</SectionBlock>
    )}

```

```

    {sections.auditoria.length > 0 && (
      <SectionBlock title="Auditoria">{renderItems(sections.auditoria)}</SectionBlock>
    )}

```

```

    {showAccountActions && sections.cuenta.length > 0 && (
      <SectionBlock title="Cuenta">{renderItems(sections.cuenta)}</SectionBlock>
    )}

```

```

    <button
      type="button"
      onClick={onLogout}
      className="w-full flex items-center justify-center gap-2 px-4 py-3 rounded-xl border border-
slate-200 text-sm font-semibold text-red-600 hover:bg-slate-50"
    >
      <Logout className="w-4 h-4" />
      Salir
    </button>
  </div>
</div>
</div>
)}

```

```

<div className="hidden md:block border-b border-slate-200 bg-white">
  <div className="mx-auto max-w-7xl px-6 py-4 flex items-center justify-between gap-4">
    <div className="min-w-0 flex-1">
      {headerLeft ? (
        headerLeft
      ) : (
        <div>
          <h1 className="text-lg font-semibold text-slate-900">{title}</h1>
          {subtitle ? <p className="text-sm text-slate-500">{subtitle}</p> : null}
        </div>
      )}
    </div>
  </div>

```

```

<div className="flex items-center gap-3 shrink-0">
  <div className="text-sm text-slate-600 hidden lg:block">{userEmail || ""}</div>

```

```

    {showAccountActions && (
      <button
        type="button"
        onClick={() => onNavigate("account")}
        className="rounded-xl border border-slate-200 bg-white px-3 py-2 text-sm font-semibold text-
slate-700 hover:bg-slate-50 transition-colors"
      >

```

```

        Mi cuenta
      </button>
    )}
  </div>
</div>
</div>

```

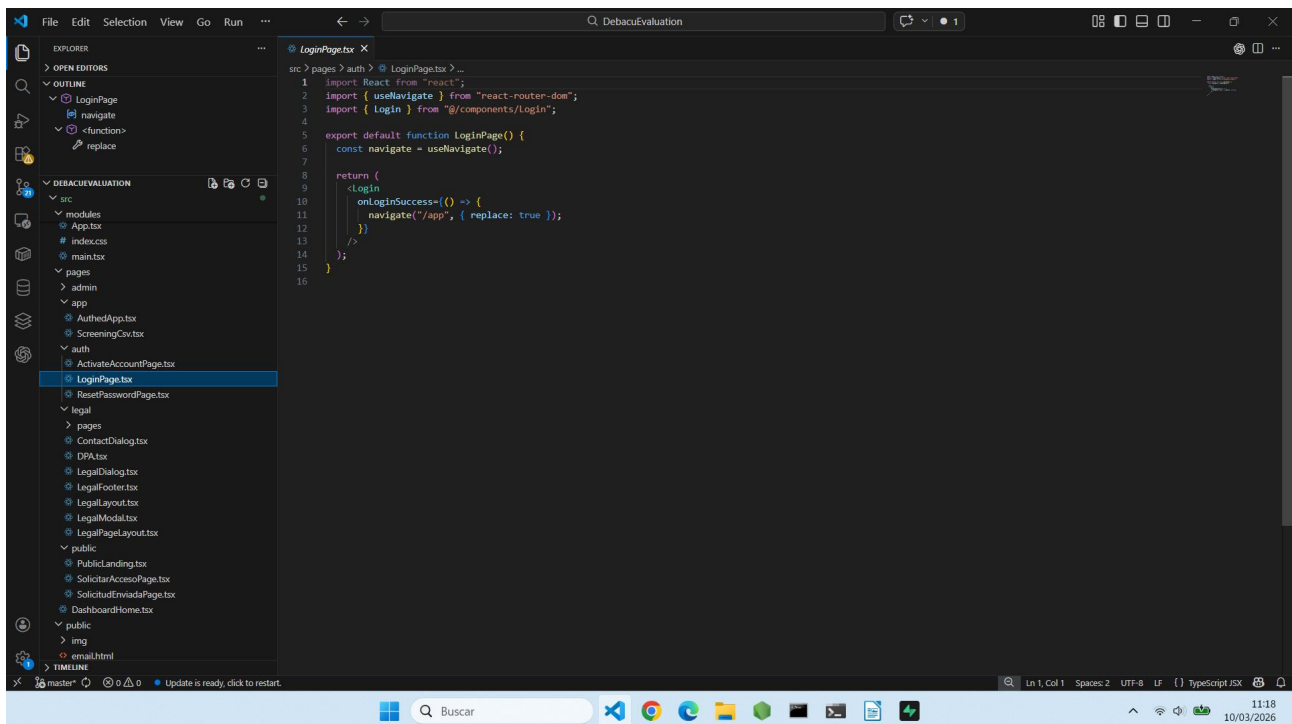
```

<main className="flex-1 overflow-auto">
  <div className="mx-auto max-w-7xl px-4 py-6 md:px-6">{children}</div>
</main>
</div>
</div>
);
}

```

en la carpeta web/ todos los ficheros nuevos (abra que quitar de aquí loginPage.tsx) ya que va en otro sitio donde esta el actual

login actual:



llama a login.tsx actual:

```

// src/pages/Login.tsx
import React, { useEffect, useMemo, useState } from "react";
import { EvalApiError, evalPostLogin } from "@services/evalApi";
import type { User } from "@types/types";
import { planCodeToPlanType, PlanType } from "@types/types";
import { Lock, User as UserIcon, Loader2 } from "lucide-react";
import { supabase } from "@services/supabaseClient";
import { useEvalAuth } from "@context/EvalAuthContext";
import { useSearchParams } from "react-router-dom";

```

```

import PaywallPlansModal from "../PaywallPlansModal";
import { setEvalOrgId } from "@services/callEvalFn";

export interface LoginProps {
  onLoginSuccess: (user: User) => void;
}

type PaywallReason = "EXPIRED" | "NONE" | null;

export const Login: React.FC<LoginProps> = ({ onLoginSuccess }) => {
  const { signIn } = useEvalAuth();
  const [searchParams] = useSearchParams();

  // Email + password
  const [emailInput, setEmailInput] = useState("");
  const [password, setPassword] = useState("");

  const [error, setError] = useState("");
  const [info, setInfo] = useState<string>("");
  const [loading, setLoading] = useState(false);

  // Paywall
  const [paywallOpen, setPaywallOpen] = useState(false);
  const [paywallReason, setPaywallReason] = useState<PaywallReason>(null);

  // evita spamear el mensaje
  const pwOk = useMemo(() => searchParams.get("pw") === "ok", [searchParams]);

  useEffect(() => {
    if (pwOk) setInfo("Contraseña actualizada. Ya puedes iniciar sesión.");
  }, [pwOk]);

  const closePaywall = () => setPaywallOpen(false);

  const openPaywall = (reason: Exclude<PaywallReason, null>) => {
    setPaywallReason(reason);
    setPaywallOpen(true);
  };

  function normalizeEmailOrThrow(v: string) {
    const email = String(v ?? "").trim().toLowerCase();
    if (!email || !email.includes("@")) throw new Error("Introduce un email válido.");
    return email;
  }

  function buildUserForUI(post: any): User {
    // post = { user, customer, membership, entitlement }
    const authUser = post?.user ?? null;
    const customer = post?.customer ?? null;
    const membership = post?.membership ?? null;
    const entitlement = post?.entitlement ?? null;
  }

```



```
// Si falta customer.id, tu UI se rompe porque User.id es customerId.  
if (!customer?.id) {  
  // En este caso, es inconsistencia (tu Edge debería devolver NO_CUSTOMER)  
  throw new Error("Login incompleto: falta customer.id en postlogin");  
}
```

```
const planCode = (entitlement?.plan_code ?? null) as any;  
const plan: PlanType = planCodeToPlanType(planCode);
```

```
const role = String(membership?.role ?? "");  
const isAdmin = role === "OWNER" || role === "ADMIN";
```

```
const email = String(authUser?.email ?? customer?.email ?? "").trim().toLowerCase();  
if (!email) throw new Error("Login incompleto: falta email");
```

```
// ⚠ En tu postlogin actual NO viene customers.name ni service_username.  
// Para no bloquear, ponemos placeholders razonables.  
// Cuando amplíes el postlogin, sustituyes esto por los campos reales.  
const fullName = String(customer?.name ?? email); // si algún día devuelves name, aquí lo aprovechas  
const username = String(customer?.service_username ?? email);
```

```
const u: User = {  
  id: String(customer.id), // ☒ customerId  
  customerId: String(customer.id), // opcional (duplicado)  
  username,  
  fullName,  
  email,  
  plan,  
  isAdmin,  
  // Si más adelante postlogin devuelve start_date o monthlyFee, los rellenas aquí  
  // planStartDate: post?.subscription?.start_date ?? undefined,  
  // monthlyFee: post?.plan?.price_monthly ?? undefined,  
};
```

```
return u;  
}
```

```
const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {  
  e.preventDefault();  
  setError("");  
  setInfo("");  
  setLoading(true);
```

```
try {  
  const email = normalizeEmailOrThrow(emailInput);
```

```
  // 1) AUTH REAL: Supabase  
  const { data, error: authError } = await supabase.auth.signInWithPassword({  
    email,  
    password,  
  });
```

```
if (authError || !data?.session) {  
  throw new Error("Usuario o contraseña incorrectos");  
}
```

```
const accessToken = data.session.access_token ?? "";  
localStorage.setItem("debacu_eval_auth_token", accessToken);
```

```
// 2) Post-login AUTHZ (Edge Function): org/membership/plan/paywall  
const post = await evalPostLogin(accessToken);
```

```
// [X] Guardar org_id para que callEvalFn lo inyecte en todas las Edge calls  
// Estructura esperada de tu postlogin: { ok:true, data:{ membership:{org_id}... } }  
const orgId =  
  (post as any)?.data?.membership?.org_id ??  
  (post as any)?.membership?.org_id ??  
  (post as any)?.data?.org_id ??  
  (post as any)?.org_id ??  
  "";
```

```
if (typeof orgId === "string" && orgId.trim()) {  
  setEvalOrgId(orgId.trim());  
}
```

```
// compat: si no existe session_token interno, guardamos ""  
localStorage.setItem("debacu_eval_session_token", String((post as any)?.session_token ?? ""));
```

```
// 3) Construir User para TU UI (customer-centric)  
const userForApp = buildUserForUI(post);
```

```
// Extra: si tu guard decide por plan, aquí ya está.  
sessionStorage.removeItem("debacu_eval_is_admin");
```

```
signIn(accessToken, userForApp);  
onLoginSuccess(userForApp);  
} catch (err: unknown) {  
  console.error(err);
```

```
if (err instanceof EvalApiError) {  
  const code = err.error_obj?.code;  
  const status = err.error_obj?.status;
```

```
if (code === "SUBSCRIPTION_NOT_ACTIVE" && status === "EXPIRED") {  
  openPaywall("EXPIRED");  
  setError("Tu periodo de prueba ha finalizado. Para continuar, contrata un plan.");  
  return;  
}
```

```
if (code === "NO_SUBSCRIPTION") {  
  openPaywall("NONE");  
  setError("No tienes una suscripción activa. Elige un plan para continuar.");  
  return;  
}
```

```

    // Si te llega NO_ORG_MEMBERSHIP, aquí NO deberías mandar a /solicitar-acceso desde Login.
    // Deja el error visible.
    setError(err.message || "Acceso denegado");
    return;
}

```

```

    setError(err instanceof Error ? err.message : "Usuario o contraseña incorrectos");
  } finally {
    setLoading(false);
  }
};

```

```

// Recuperación contraseña vía Supabase (email-only)
const handleForgotPassword = async () => {
  setError("");
  setInfo("");

```

```

  const raw = emailInput.trim();
  if (!raw) {
    setError("Introduce tu email para recuperar la contraseña.");
    return;
  }

```

```

  let email = "";
  try {
    email = normalizeEmailOrThrow(raw);
  } catch {
    setError("Introduce un email válido para recuperar la contraseña.");
    return;
  }

```

```

  setLoading(true);
  try {
    const redirectTo = `${window.location.origin}/reset-password`;

```

```

    const { error: e } = await supabase.auth.resetPasswordForEmail(email, { redirectTo });
    if (e) throw e;

```

```

    // anti-enumeración
    setInfo("Si el email existe, recibirás un enlace para restablecer la contraseña en unos minutos.");
  } catch (e: any) {
    // anti-enumeración
    setInfo("Si el email existe, recibirás un enlace para restablecer la contraseña en unos minutos.");
    console.warn("resetPasswordForEmail failed:", e?.message ?? e);
  } finally {
    setLoading(false);
  }
};

```

```

const showPlansButton = paypalReason !== null;

```

```

return (
  <div className="relative min-h-screen flex items-center justify-center overflow-hidden">
    <div className="absolute inset-0 bg-gradient-to-br from-[#06213f] via-[#0b3a6f] to-[#0e4f8a]" />
    <div className="absolute -top-40 -right-40 w-[520px] h-[520px] rounded-full bg-white/5 blur-3xl" />
    <div className="absolute -bottom-40 -left-40 w-[520px] h-[520px] rounded-full bg-black/10 blur-3xl"
  />

```

```

    <div className="relative z-10 w-full max-w-md p-8 bg-white rounded-2xl shadow-2xl border border-
slate-200">
      <div className="text-center mb-8">
        <div className="mx-auto mb-4 flex h-14 w-14 items-center justify-center rounded-full bg-indigo-
100 text-indigo-600">
          <Lock className="h-7 w-7" />
        </div>

```

```

    <h1 className="text-2xl font-bold text-slate-900">DebacuEvaluation360</h1>

```

```

    <p className="mt-2 text-sm text-slate-600">
      Plataforma profesional de evaluación y control operativo
    </p>

```

```

    <p className="mt-3 text-xs text-slate-400">
      Acceso restringido · Uso profesional · No es una plataforma pública
    </p>
  </div>

```

```

  <form onSubmit={handleSubmit} className="space-y-6">
    {info && (
      <div className="rounded-lg border border-emerald-100 bg-emerald-50 px-4 py-3 text-sm text-
emerald-700">
        {info}
      </div>
    )}
  </form>

```

```

  {error && (
    <div className="rounded-lg border border-red-100 bg-red-50 px-4 py-3 text-sm text-red-600">
      {error}
    </div>
  )}

```

```

  <div>
    <label className="mb-1 block text-sm font-medium text-slate-700">Email</label>
    <div className="relative">
      <UserIcon className="absolute left-3 top-1/2 h-5 w-5 -translate-y-1/2 text-slate-400" />
      <input
        type="email"
        required
        value={emailInput}
        onChange={(e) => setEmailInput(e.target.value)}
        className="w-full rounded-lg border border-slate-300 py-2.5 pl-10 pr-3 text-sm focus:border-
indigo-500 focus:ring-indigo-500"
        placeholder="tuemail@dominio.com"
        autoComplete="username"
      />
    </div>
  </div>

```

```
    />
  </div>
```

```
    <div className="mt-2 flex items-center justify-end">
      <button
        type="button"
        onClick={handleForgotPassword}
        disabled={loading}
        className="text-xs font-semibold text-indigo-700 hover:text-indigo-800 disabled:opacity-50"
        title="Envía un enlace de recuperación al email indicado"
      >
        ¿Olvidaste tu contraseña?
      </button>
    </div>
  </div>
```

```
    <div>
      <label className="mb-1 block text-sm font-medium text-slate-700">Contraseña</label>
      <div className="relative">
        <Lock className="absolute left-3 top-1/2 h-5 w-5 -translate-y-1/2 text-slate-400" />
        <input
          type="password"
          required
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="w-full rounded-lg border border-slate-300 py-2.5 pl-10 pr-3 text-sm focus:border-indigo-500 focus:ring-indigo-500"
          placeholder="••••••••"
          autoComplete="current-password"
        />
      </div>
    </div>
```

```
    <button
      type="submit"
      disabled={loading}
      className="flex w-full items-center justify-center gap-2 rounded-lg bg-indigo-600 py-2.5 text-sm font-semibold text-white hover:bg-indigo-700 disabled:opacity-50"
    >
      {loading && <Loader2 className="h-5 w-5 animate-spin" />}
      Acceder
    </button>
```

```
    {showPlansButton && (
      <button
        type="button"
        onClick={() => setPaywallOpen(true)}
        className="w-full rounded-lg border border-slate-200 bg-white py-2.5 text-sm font-semibold text-slate-700 hover:bg-slate-50"
      >
        Ver planes
      </button>
    )}
```

```

    </form>
  </div>

  { /* Paywall Modal (JWT-only) */ }
  {paywallOpen && (
    <PaywallPlansModal open={paywallOpen} onClose={closePaywall} reason={paywallReason} />
  )}
</div>
);
};

```

mas ficheros AuthContext.tsx

```

import React, { createContext, useContext, useEffect, useMemo, useState } from "react";
import { supabase } from "@services/supabaseClient";
import type { Session, User } from "@supabase/supabase-js";

```

```

type AuthState = {
  session: Session | null;
  user: User | null;
  loading: boolean;
  signOut: () => Promise<void>;
  refreshSession: () => Promise<void>;
};

```

```

const AuthContext = createContext<AuthState | undefined>(undefined);

```

```

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [session, setSession] = useState<Session | null>(null);
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

```

```

  const refreshSession = async () => {
    const { data, error } = await supabase.auth.getSession();
    if (error) {
      console.error("getSession error:", error);
      setSession(null);
      setUser(null);
      return;
    }
    setSession(data.session ?? null);
    setUser(data.session?.user ?? null);
  };

```

```

  useEffect(() => {
    let mounted = true;

```

```

    const boot = async () => {
      setLoading(true);
      await refreshSession();
      if (mounted) setLoading(false);
    };

```

```

    void boot();

    const { data: sub } = supabase.auth.onAuthStateChange((_event, newSession) => {
      setSession(newSession);
      setUser(newSession?.user ?? null);
      setLoading(false);
    });

    return () => {
      mounted = false;
      sub.subscription.unsubscribe();
    };
  }, []);

  const signOut = async () => {
    await supabase.auth.signOut();
    setSession(null);
    setUser(null);
  };

  const value = useMemo<AuthState>(
    () => ({ session, user, loading, signOut, refreshSession }),
    [session, user, loading]
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within <AuthProvider />");
  return ctx;
}

```

y este otro: EvalAuthContext.tsx

```

import React, { createContext, useContext, useEffect, useMemo,
useState } from "react";

```

```

import { supabase } from "@services/supabaseClient";
import type { User } from "@types/types";

```

```

const TOKEN_KEY = "debacu_eval_token";
const USER_KEY = "debacu_eval_user";

```

```

type EvalAuthState = {
  token: string | null;
  user: User | null;
  loading: boolean;
  signIn: (token: string, user: User) => void;
  signOut: () => Promise<void>;
  updateUser: (user: User) => void;
};

```

```

const EvalAuthContext = createContext<EvalAuthState | undefined>(undefined);

```

```

export function EvalAuthProvider({ children }: { children: React.ReactNode }) {

```

```
const [token, setToken] = useState<string | null>(null);
const [user, setUser] = useState<User | null>(null);
const [loading, setLoading] = useState(true);
```

```
useEffect(() => {
  let mounted = true;
```

```
const bootstrap = async () => {
  setLoading(true);
  try {
    const { data } = await supabase.auth.getSession();
    if (!mounted) return;
    const storedToken = data.session?.access_token ?? null;
    const storedUser = localStorage.getItem(USER_KEY);
```

```
    setToken(storedToken);
    setUser(storedUser ? (JSON.parse(storedUser) as User) : null);
  } catch (error) {
    console.error("EvalAuth bootstrap error:", error);
    if (!mounted) return;
    setToken(null);
    setUser(null);
  } finally {
    if (mounted) setLoading(false);
  }
};
```

```
bootstrap();
```

```
const { data: subscription } = supabase.auth.onAuthStateChange((_event, session) => {
  setToken(session?.access_token ?? null);
  if (!session) {
    setUser(null);
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
  }
});
```

```
return () => {
  mounted = false;
  subscription.subscription.unsubscribe();
};
}, []);
```

```
const signIn = (t: string, u: User) => {
  setToken(t);
  setUser(u);
  localStorage.setItem(TOKEN_KEY, t);
  localStorage.setItem(USER_KEY, JSON.stringify(u));
};
```

```
const updateUser = (u: User) => {
```



```

    setUser(u);
    localStorage.setItem(USER_KEY, JSON.stringify(u));
  };

const signOut = async () => {
  await supabase.auth.signOut();
  setToken(null);
  setUser(null);
  localStorage.removeItem(TOKEN_KEY);
  localStorage.removeItem(USER_KEY);
};

const value = useMemo<EvalAuthState>(
  () => ({
    token,
    user,
    loading,
    signIn,
    signOut,
    updateUser,
  }),
  [token, user, loading, signIn, signOut, updateUser]
);

return <EvalAuthContext.Provider value={value}>{children}</EvalAuthContext.Provider>;
}

export function useEvalAuth() {
  const ctx = useContext(EvalAuthContext);
  if (!ctx) throw new Error("useEvalAuth must be used within <EvalAuthProvider />");
  return ctx;
}

```

están en la carpeta components/context

en cuanto a los ficheros nuevos ya te he dicho dónde están ubicados no sé si las imágenes que muestran se descargan de la web o después no las tendremos, te paso una de ejemplo

```

import React,
{ useState } from 'react';
import { motion } from 'motion/react';
import {
  CheckCircle2,
  ArrowLeft,
  Building2,
  User,
  Activity,
  ShieldCheck,
  AlertCircle,
  Loader2
} from 'lucide-react';
import { cn } from '../lib/utils';

interface FormData {
  firstName: string;

```

```
lastName: string;
role: string;
email: string;
phone: string;
website: string;
hotelName: string;
legalName: string;
taxId: string;
accommodationType: string;
roomCount: string;
employeeCount: string;
address: string;
postalCode: string;
city: string;
province: string;
country: string;
pms: string;
revenueAnalysisStatus: string;
mainInterest: string;
monthlyBookings: string;
comments: string;
privacyAccepted: boolean;
representationConfirmed: boolean;
manualReviewAccepted: boolean;
}
```

```
const initialFormData: FormData = {
  firstName: '',
  lastName: '',
  role: '',
  email: '',
  phone: '',
  website: '',
  hotelName: '',
  legalName: '',
  taxId: '',
  accommodationType: '',
  roomCount: '',
  employeeCount: '',
  address: '',
  postalCode: '',
  city: '',
  province: '',
  country: '',
  pms: '',
  revenueAnalysisStatus: '',
  mainInterest: '',
  monthlyBookings: '',
  comments: '',
  privacyAccepted: false,
  representationConfirmed: false,
  manualReviewAccepted: false,
};
```

```

export const AccessRequestPage = ({ onBack }: { onBack: () => void }) => {
  const [formData, setFormData] = useState<FormData>(initialFormData);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [isSuccess, setIsSuccess] = useState(false);
  const [errors, setErrors] = useState<Partial<Record<keyof FormData, string>>>({});

  const validate = () => {
    const newErrors: Partial<Record<keyof FormData, string>> = {};
    if (!formData.firstName) newErrors.firstName = 'El nombre es obligatorio';
    if (!formData.lastName) newErrors.lastName = 'Los apellidos son obligatorios';
    if (!formData.email) {
      newErrors.email = 'El email es obligatorio';
    } else if (!/^\S+@\S+\.\S+/.test(formData.email)) {
      newErrors.email = 'El formato del email no es válido';
    }
    if (!formData.phone) newErrors.phone = 'El teléfono es obligatorio';
    if (!formData.hotelName) newErrors.hotelName = 'El nombre del alojamiento es obligatorio';
    if (!formData.taxId) newErrors.taxId = 'El CIF/NIF es obligatorio';
    if (!formData.accommodationType) newErrors.accommodationType = 'El tipo de alojamiento es obligatorio';
    if (!formData.roomCount) newErrors.roomCount = 'El número de habitaciones es obligatorio';
    if (!formData.city) newErrors.city = 'La ciudad es obligatoria';
    if (!formData.country) newErrors.country = 'El país es obligatorio';
    if (!formData.privacyAccepted) newErrors.privacyAccepted = 'Debes aceptar la política de privacidad';

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!validate()) return;

    setIsSubmitting(true);
    try {
      const response = await fetch('/api/access-requests', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });
    }

    if (response.ok) {
      setIsSuccess(true);
    } else {
      alert('Error al enviar la solicitud. Por favor, inténtalo de nuevo.');
```

```

    }
    catch (error) {
      console.error('Error submitting form:', error);
      alert('Error de conexión. Por favor, inténtalo de nuevo.');
```

```

    } finally {
      setIsSubmitting(false);
    }
  }
}
```

```

    });

    const handleChange = (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>)
=> {
    const { name, value, type } = e.target;
    const checked = (e.target as HTMLInputElement).checked;

    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));

    if (errors[name as keyof FormData]) {
      setErrors(prev => ({ ...prev, [name]: undefined }));
    }
  });

  if (isSuccess) {
    return (
      <div className="min-h-screen flex items-center justify-center px-6 py-20 bg-[#020617]">
        <motion.div
          initial={{ opacity: 0, scale: 0.9 }}
          animate={{ opacity: 1, scale: 1 }}
          className="max-w-xl w-full glass-card p-12 text-center border-emerald-500/20 bg-emerald-500/5"
        >
          <div className="w-20 h-20 rounded-full bg-emerald-500/10 flex items-center justify-center mx-auto
mb-8 text-emerald-500">
            <CheckCircle2 size={48} />
          </div>
          <h2 className="text-3xl font-display font-bold text-white mb-4">Solicitud recibida
correctamente</h2>
          <p className="text-slate-400 mb-12 leading-relaxed">
            Hemos recibido tu solicitud de acceso. Nuestro equipo revisará la información del alojamiento y,
si la validación es correcta, recibirás un email con la invitación para activar tu cuenta y crear tu
contraseña.
          </p>
          <button
            onClick={onBack}
            className="btn-primary w-full py-4"
          >
            Volver al inicio
          </button>
        </motion.div>
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-[#020617] pt-32 pb-20 px-6">
      <div className="max-w-4xl mx-auto">
        <button
          onClick={onBack}
          className="flex items-center gap-2 text-slate-500 hover:text-white transition-colors mb-8 group"

```

```

    <ArrowLeft size={18} className="group-hover:-translate-x-1 transition-transform" />
    Volver
  </button>

```

```

    <div className="mb-12">
      <h1 className="text-4xl md:text-5xl font-display font-bold text-white mb-4">Solicitar acceso a
      Debacu</h1>
      <p className="text-slate-400 text-lg">
        Completa el formulario para que podamos validar tu alojamiento y habilitar el acceso a la
        plataforma.
      </p>
      <div className="mt-6 p-4 bg-blue-500/10 border border-blue-500/20 rounded-xl flex items-start
      gap-3 text-sm text-blue-400">
        <ShieldCheck size={20} className="shrink-0" />
        <p>Debacu revisa cada solicitud manualmente para verificar que corresponde a un alojamiento real
        o empresa del sector hospitality.</p>
      </div>
    </div>

```

```

    <form onSubmit={handleSubmit} className="space-y-12">
      {/* Bloque 1: Datos de contacto */}
      <div className="glass-card p-8 border-white/[0.05]">
        <div className="flex items-center gap-3 mb-8">
          <div className="w-10 h-10 rounded-lg bg-blue-600/10 flex items-center justify-center text-blue-
          500">
            <User size={20} />
          </div>
          <h3 className="text-xl font-bold text-white">Datos de contacto</h3>
        </div>
        <div className="grid md:grid-cols-2 gap-6">
          <div className="space-y-2">
            <label className="text-xs font-bold text-slate-500 uppercase">Nombre *</label>
            <input
              type="text"
              name="firstName"
              value={formData.firstName}
              onChange={handleChange}
              className={cn(
                "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
                500/50 transition-colors",
                errors.firstName ? "border-red-500/50" : "border-white/10"
              )}
              placeholder="Tu nombre"
            />
            {errors.firstName && <p className="text-[10px] text-red-500">{errors.firstName}</p>}
          </div>
          <div className="space-y-2">
            <label className="text-xs font-bold text-slate-500 uppercase">Apellidos *</label>
            <input
              type="text"
              name="lastName"

```

```

        value={formData.lastName}
        onChange={handleChange}
        className={cn(
            "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
500/50 transition-colors",
            errors.lastName ? "border-red-500/50" : "border-white/10"
        )}
        placeholder="Tus apellidos"
    />
    {errors.lastName && <p className="text-[10px] text-red-500">{errors.lastName}</p>}
</div>
<div className="space-y-2">
    <label className="text-xs font-bold text-slate-500 uppercase">Cargo / Puesto</label>
    <input
        type="text"
        name="role"
        value={formData.role}
        onChange={handleChange}
        className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
        placeholder="Ej: Director de Operaciones"
    />
</div>
<div className="space-y-2">
    <label className="text-xs font-bold text-slate-500 uppercase">Email Profesional *</label>
    <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        className={cn(
            "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
500/50 transition-colors",
            errors.email ? "border-red-500/50" : "border-white/10"
        )}
        placeholder="tu@empresa.com"
    />
    {errors.email && <p className="text-[10px] text-red-500">{errors.email}</p>}
</div>
<div className="space-y-2">
    <label className="text-xs font-bold text-slate-500 uppercase">Teléfono *</label>
    <input
        type="tel"
        name="phone"
        value={formData.phone}
        onChange={handleChange}
        className={cn(
            "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
500/50 transition-colors",
            errors.phone ? "border-red-500/50" : "border-white/10"
        )}
        placeholder="+34 000 000 000"
    />

```

```

        {errors.phone && <p className="text-[10px] text-red-500">{errors.phone}</p>}
      </div>
      <div className="space-y-2">
        <label className="text-xs font-bold text-slate-500 uppercase">Web del hotel o empresa</label>
        <input
          type="url"
          name="website"
          value={formData.website}
          onChange={handleChange}
          className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
          placeholder="https://www.tu-hotel.com"
        />
      </div>
    </div>
  </div>

```

```

    {/* Bloque 2: Datos del alojamiento */}
    <div className="glass-card p-8 border-white/[0.05]">
      <div className="flex items-center gap-3 mb-8">
        <div className="w-10 h-10 rounded-lg bg-emerald-600/10 flex items-center justify-center text-
emerald-500">
          <Building2 size={20} />
        </div>
        <h3 className="text-xl font-bold text-white">Datos del alojamiento o empresa</h3>
      </div>
      <div className="grid md:grid-cols-2 gap-6">
        <div className="space-y-2 md:col-span-2">
          <label className="text-xs font-bold text-slate-500 uppercase">Nombre comercial del hotel /
alojamiento *</label>
          <input
            type="text"
            name="hotelName"
            value={formData.hotelName}
            onChange={handleChange}
            className={cn(
              "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
500/50 transition-colors",
              errors.hotelName ? "border-red-500/50" : "border-white/10"
            )}
            placeholder="Nombre del hotel"
          />
          {errors.hotelName && <p className="text-[10px] text-red-500">{errors.hotelName}</p>}
        </div>
        <div className="space-y-2">
          <label className="text-xs font-bold text-slate-500 uppercase">Razón Social</label>
          <input
            type="text"
            name="legalName"
            value={formData.legalName}
            onChange={handleChange}

```

```

        className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
        placeholder="Nombre legal de la empresa"
      />
    </div>
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">CIF / NIF Empresa *</label>
      <input
        type="text"
        name="taxId"
        value={formData.taxId}
        onChange={handleChange}
        className={cn(
          "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
500/50 transition-colors",
          errors.taxId ? "border-red-500/50" : "border-white/10"
        )}
        placeholder="B000000000"
      />
      {errors.taxId && <p className="text-[10px] text-red-500">{errors.taxId}</p>}
    </div>
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">Tipo de alojamiento *</label>
      <select
        name="accommodationType"
        value={formData.accommodationType}
        onChange={handleChange}
        className={cn(
          "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-
500/50 transition-colors appearance-none",
          errors.accommodationType ? "border-red-500/50" : "border-white/10"
        )}
      >
        <option value="" className="bg-slate-900">Seleccionar...</option>
        <option value="hotel" className="bg-slate-900">Hotel</option>
        <option value="apartahotel" className="bg-slate-900">Apartahotel</option>
        <option value="apartamentos" className="bg-slate-900">Apartamentos turísticos</option>
        <option value="rural" className="bg-slate-900">Casa rural</option>
        <option value="hostal" className="bg-slate-900">Hostal</option>
        <option value="otro" className="bg-slate-900">Otro</option>
      </select>
      {errors.accommodationType && <p className="text-[10px] text-red-
500">{errors.accommodationType}</p>}
    </div>
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">Número de habitaciones
*</label>
      <input
        type="number"
        name="roomCount"
        value={formData.roomCount}
        onChange={handleChange}
        className={cn(

```



```

        "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-500/50 transition-colors",
        errors.roomCount ? "border-red-500/50" : "border-white/10"
    )}
    placeholder="0"
  />
  {errors.roomCount && <p className="text-[10px] text-red-500">{errors.roomCount}</p>}}
</div>
<div className="space-y-2">
  <label className="text-xs font-bold text-slate-500 uppercase">Número estimado de
empleados</label>
  <input
    type="number"
    name="employeeCount"
    value={formData.employeeCount}
    onChange={handleChange}
    className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
    placeholder="0"
  />
</div>
<div className="space-y-2 md:col-span-2">
  <label className="text-xs font-bold text-slate-500 uppercase">Dirección</label>
  <input
    type="text"
    name="address"
    value={formData.address}
    onChange={handleChange}
    className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
    placeholder="Calle, número, planta"
  />
</div>
<div className="space-y-2">
  <label className="text-xs font-bold text-slate-500 uppercase">Código Postal</label>
  <input
    type="text"
    name="postalCode"
    value={formData.postalCode}
    onChange={handleChange}
    className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
    placeholder="00000"
  />
</div>
<div className="space-y-2">
  <label className="text-xs font-bold text-slate-500 uppercase">Ciudad *</label>
  <input
    type="text"
    name="city"
    value={formData.city}
    onChange={handleChange}
    className={cn(

```

```

        "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-500/50 transition-colors",
        errors.city ? "border-red-500/50" : "border-white/10"
    )}
    placeholder="Ciudad"
  />
  {errors.city && <p className="text-[10px] text-red-500">{errors.city}</p>}
</div>
<div className="space-y-2">
  <label className="text-xs font-bold text-slate-500 uppercase">Provincia / Estado</label>
  <input
    type="text"
    name="province"
    value={formData.province}
    onChange={handleChange}
    className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
    placeholder="Provincia"
  />
</div>
<div className="space-y-2">
  <label className="text-xs font-bold text-slate-500 uppercase">País *</label>
  <input
    type="text"
    name="country"
    value={formData.country}
    onChange={handleChange}
    className={cn(
      "w-full bg-white/5 border rounded-lg p-3 text-sm focus:outline-none focus:border-blue-500/50 transition-colors",
      errors.country ? "border-red-500/50" : "border-white/10"
    )}
    placeholder="España"
  />
  {errors.country && <p className="text-[10px] text-red-500">{errors.country}</p>}
</div>
</div>
</div>

```

```

{ /* Bloque 3: Información operativa */ }
<div className="glass-card p-8 border-white/[0.05]">
  <div className="flex items-center gap-3 mb-8">
    <div className="w-10 h-10 rounded-lg bg-violet-600/10 flex items-center justify-center text-
violet-500">
      <Activity size={20} />
    </div>
    <h3 className="text-xl font-bold text-white">Información operativa</h3>
  </div>
  <div className="grid md:grid-cols-2 gap-6">
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">PMS Actual</label>
      <input

```

```

        type="text"
        name="pms"
        value={formData.pms}
        onChange={handleChange}
        className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
        placeholder="Ej: Opera, Mews, Cloudbeds..."
      />
    </div>
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">¿Trabaja con análisis de
revenue?</label>
      <select
        name="revenueAnalysisStatus"
        value={formData.revenueAnalysisStatus}
        onChange={handleChange}
        className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50 appearance-none"
      >
        <option value="" className="bg-slate-900">Seleccionar...</option>
        <option value="si" className="bg-slate-900">Sí</option>
        <option value="no" className="bg-slate-900">No</option>
        <option value="parcialmente" className="bg-slate-900">Parcialmente</option>
      </select>
    </div>
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">Interés principal en
Deba<div>
      <select
        name="mainInterest"
        value={formData.mainInterest}
        onChange={handleChange}
        className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50 appearance-none"
      >
        <option value="" className="bg-slate-900">Seleccionar...</option>
        <option value="riesgo" className="bg-slate-900">Riesgo operativo</option>
        <option value="revenue" className="bg-slate-900">Revenue intelligence</option>
        <option value="trazabilidad" className="bg-slate-900">Control y trazabilidad</option>
        <option value="ia" className="bg-slate-900">IA aplicada a la operativa</option>
        <option value="otro" className="bg-slate-900">Otro</option>
      </select>
    </div>
    <div className="space-y-2">
      <label className="text-xs font-bold text-slate-500 uppercase">Reservas mensuales
estimadas</label>
      <input
        type="text"
        name="monthlyBookings"
        value={formData.monthlyBookings}
        onChange={handleChange}
        className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
        placeholder="Ej: 500 - 1000"

```

```

    />
  </div>
  <div className="space-y-2 md:col-span-2">
    <label className="text-xs font-bold text-slate-500 uppercase">Observaciones o
comentarios</label>
    <textarea
      name="comments"
      value={formData.comments}
      onChange={handleChange}
      rows={4}
      className="w-full bg-white/5 border border-white/10 rounded-lg p-3 text-sm focus:outline-
none focus:border-blue-500/50"
      placeholder="Cuéntanos más sobre tus necesidades..."
    ></textarea>
  </div>
</div>
</div>

```

```

{ /* Bloque 4: Verificación */ }
<div className="glass-card p-8 border-white/[0.05] space-y-6">
  <div className="space-y-4">
    <label className="flex items-start gap-3 cursor-pointer group">
      <input
        type="checkbox"
        name="representationConfirmed"
        checked={formData.representationConfirmed}
        onChange={handleChange}
        className="mt-1 w-4 h-4 rounded border-white/10 bg-white/5 text-blue-600 focus:ring-0
focus:ring-offset-0"
      />
      <span className="text-sm text-slate-400 group-hover:text-slate-300 transition-colors">Confirmo
que represento al alojamiento o empresa indicada</span>
    </label>
    <label className="flex items-start gap-3 cursor-pointer group">
      <input
        type="checkbox"
        name="privacyAccepted"
        checked={formData.privacyAccepted}
        onChange={handleChange}
        className={cn(
          "mt-1 w-4 h-4 rounded border-white/10 bg-white/5 text-blue-600 focus:ring-0 focus:ring-
offset-0",
          errors.privacyAccepted && "border-red-500/50"
        )}
      />
      <span className="text-sm text-slate-400 group-hover:text-slate-300 transition-colors">Acepto
la <button type="button" className="text-blue-500 hover:underline">política de privacidad</button>
*</span>
    </label>
    {errors.privacyAccepted && <p className="text-[10px] text-red-500 ml-
7">{errors.privacyAccepted}</p>}}
  </div>
  <label className="flex items-start gap-3 cursor-pointer group">

```

```

☒


```

```

<div className="pt-6 border-t border-white/[0.05]">
  <button
    type="submit"
    disabled={isSubmitting}
    className="btn-primary w-full py-4 flex items-center justify-center gap-2 disabled:opacity-50
disabled:cursor-not-allowed"
  >
    {isSubmitting ? (
      <>
        <Loader2 size={20} className="animate-spin" />
        Enviando solicitud...
      </>
    ) : (
      'Enviar solicitud'
    )}
  </button>
  <p className="text-[10px] text-slate-600 text-center mt-4">
    Debacu revisa manualmente las solicitudes para proteger la integridad de la plataforma y
    garantizar que el acceso se concede únicamente a empresas y alojamientos reales del sector.
  </p>
</div>
</div>
</form>
</div>
</div>
);
};

```

veo que usan algo de motion/react ?????